

MEMAGENT: RESHAPING LONG-CONTEXT LLM WITH MULTI-CONV RL-BASED MEMORY AGENT

Hongli Yu^{1,2,3} Tinghong Chen¹ Jiangtao Feng¹ Jiangjie Chen^{2,3} Weinan Dai^{1,2,3}
 Qiying Yu^{1,2,3} Ya-Qin Zhang^{1,3} Wei-Ying Ma^{1,3} Jingjing Liu^{1,3} Mingxuan Wang^{2,3*}

Hao Zhou^{1,3*}

¹ Institute for AI Industry Research (AIR), Tsinghua University

² ByteDance Seed

³ SIA-Lab of Tsinghua AIR and ByteDance Seed

ABSTRACT

Despite improvements by length extrapolation, efficient attention and memory modules, handling infinitely long documents without performance degradation during extrapolation remains the ultimate challenge in long-text processing. To solve this problem, we introduce a novel agent workflow, MEMAGENT, which processes text in segments and updates memory through an overwrite strategy, addressing the challenge of long-context task through enhanced memory management. We further extend the DAPO algorithm to directly optimize memory ability in an end-to-end fashion, facilitating training via independent-context multi-conversation generation. Experimental results demonstrate that MEMAGENT has superb long-context capabilities, being able to extrapolate from an 8K context to a 3.5M QA task with a performance loss of less than 10% and achieving over 95% on the 512K NIAH test.

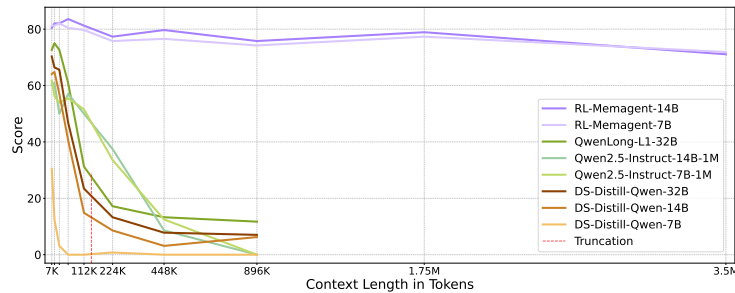


Figure 1: Accuracy scores of RULER-HQA (Hsieh et al., 2024; Yang et al., 2018). Even models that employ long-context continual pretraining and extrapolation techniques fail to maintain consistent performance. In contrast, MEMAGENT with RL only demonstrates marginal performance dropping.

1 INTRODUCTION

While having demonstrated impressive capabilities (OpenAI, 2024; DeepMind, 2024; XAI, 2024; Anthropic, 2024; OpenAI, 2023), industry-level Large Language Model (LLM) systems (Anthropic, 2025; Li et al., 2025a; Liu et al., 2024; Yen et al., 2024) still face a critical challenge: how to handle long contexts effectively - processing an entire book, executing a complex chain of reasoning over many steps, or managing the long-term memory of an agent system - all these complex tasks can generate overflowing text that quickly explodes the typical-size context window of current LLMs.

**Corresponding author.

MEMAGENTMEMAGENT: 重塑长上下文大语言模型的多卷积RL基础记忆代理

Hongli Yu^{1,2,3} Tinghong Chen¹ Jiangtao Feng¹ Jiangjie Chen^{2,3} Weinan Dai^{1,2,3}

Qiying Yu^{1,2,3} Ya-Qin Zhang^{1,3} Wei-Ying Ma^{1,3} Jingjing Liu^{1,3} Mingxuan Wang^{2,3*}

Hao Zhou^{1,3*}

¹ 清华大学人工智能产业研究院 (AIR) ² 字节跳动种子计

划³ 清华大学AIR和字节跳动种子计划SIA实验室

摘要

尽管通过长度外推、高效注意力和内存模块进行了改进，但在外推过程中处理无限长文档而不出现性能下降仍然是长文本处理中的终极挑战。为了解决这个问题，我们引入了一种新的代理工作流程MEMAGENT，该工作流程通过覆盖策略分段处理文本并更新内存，通过增强内存管理来解决长上下文任务的挑战。我们进一步将DAPO算法扩展为以端到端的方式直接优化内存能力，通过独立上下文多对话生成来促进训练。实验结果表明，MEMAGENT具有出色的长上下文能力，能够从8K上下文外推到3.5M问答任务，性能损失小于10%，并在512K NIAH测试中超过95%。

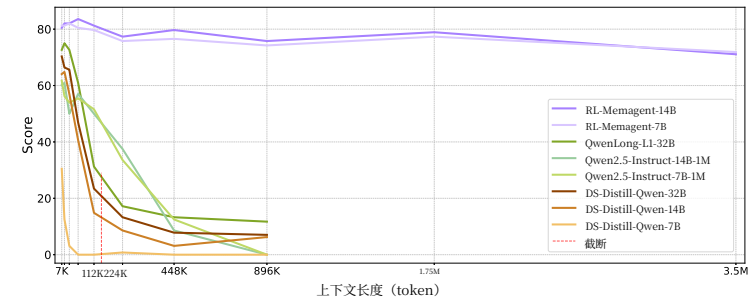


图1: RULER-HQA的准确率得分 (Hsieh等人, 2024; Yang等人, 2018)。即便是采用长上下文持续预训练和外推技术的模型也无法保持稳定的性能。相比之下，仅使用RL的MEMAGENT仅表现出轻微的性能下降。

1 引言

虽然已经展示了令人印象深刻的性能 (OpenAI, 2024; DeepMind, 2024; XAI, 2024; Anthropic, 2024; OpenAI, 2023)，行业级的大语言模型 (LLM) 系统 (Anthropic, 2025; Li 等人, 2025a; Liu 等人, 2024; Yen 等人, 2024) 仍然面临一个关键挑战：如何有效处理长上下文——处理整本书、执行多步复杂推理链，或管理智能体系统的长期记忆——所有这些复杂任务都可能生成大量文本，迅速超出当前 LLMs 的典型大小上下文窗口。

**通讯作者。

Existing approaches to long-context tasks are three-pronged. The first involves length extrapolation methods by shifting the positional embeddings in order to extend the context window of the model (Su et al., 2024; bloc97, 2023; Chen et al., 2023; Peng et al., 2023b; An et al., 2024), plus continued pre-training (Liu et al., 2023; Xiong et al., 2023; Gao et al., 2025). Despite promising potential, these methods often suffer from performance degradation and slow processing speed due to $O(n^2)$ computational complexity when applied to extremely long text. The second school of methods leverages sparse attention (Beltagy et al., 2020; Zhao et al., 2019; Xiao et al., 2023) and linear attention mechanisms (Child et al., 2019; Katharopoulos et al., 2020) to reduce the complexity of attention for more efficient processing of longer sequences. However, this typically requires training from scratch, with inherent adversities such as linear attention facing difficulties in parallel training or sparse attention depending on human-defined patterns. The last line of inquiry investigates context compression (Jiang et al., 2023; Li et al., 2023; Behrouz et al., 2024; Zhang et al., 2024), which aims to condense information in token-level or external-memory-plugin modules. Such approaches often struggle with extrapolation, and require the integration of additional modules or context operations, which ineluctably disrupts the standard generation process and hinders compatibility as well as parallelization.

Hence, a successful LLM with strong long-context capabilities requires the trinity of: 1) processing infinite length of text; 2) scaling without significant performance drop; and 3) efficient decoding with linear complexity. To pursue this quest, we return to the basic intuition behind long-context modeling (Miller et al., 1956; Hochreiter & Schmidhuber, 1997; Graves et al., 2014; Weston et al., 2014). When humans process long-context information, we tend to abstract out the main revealing conceptions to capture the essence of the whole text, often by making notes of critical details or using short-handed stenograph to record the key points, while discarding redundant and irrelevant data. We do not attempt to memorize every single fact or each small piece of information; instead, we focus our intellectual energy on more important aspects of the task at hand. This selective attention not only simplifies the process but also aids in tackling complex problems more efficiently.

Following this anthropocentric intuition, we propose a novel use of Reinforcement Learning (RL) to equip LLMs with a dynamically updated fixed-length ‘memory’, as illustrated in Figure 2. During inference, the LLM processes the input text segment-by-segment. As it reads each segment, the model proactively and selectively updates the memory, which then contributes to the generation of the final output after all relevant messages are aggregated and synergized in the memory. This clever mechanism allows the LLM to flexibly handle arbitrary text lengths while maintaining a linear time complexity during processing, since the length of the memory is fixed, which leads to a fixed context window size for the model. This segment-based approach generates multiple outputs from a single long-text input, requiring multiple rounds of memory updates and a final round for the generation of the final response. Training this type of agent workflow, which enables dialogues across multiple independent contexts, is still an unexplored territory in current LLM study. Existing systems typically handle workflow trajectories via alternating tool calls or environment feedback by either simply concatenating (Ouyang et al., 2025; Jin et al., 2025) them or using a sliding window (Feng et al., 2025) approach, which lacks flexibility and scalability in practice. Our MEMAGENT approach, instead, proposes that treats each context-independent conversation as an optimization objective. Based on the DAPO (Yu et al., 2025) algorithm, we implement the Multi-Conv DAPO to optimize an arbitrary agent workflow by verifiable outcome reward.

In our experiments, an RL-trained model with a modest 8K context window (with a 1024-token memory and a 5000-token document chunk) trained on 60K length documents exhibits consistently superb capabilities for Question Answering (QA) tasks on documents of up to 3.5 million tokens, without performance drop and with linear computation cost. This demonstratively showcases the efficiency and scalability of our long-context memory approach.

Our major contributions are threefold:

- We introduce a novel approach that enables LLMs to process arbitrarily long inputs within limited context window under linear time complexity during inference, overcoming a significant bottleneck in long-context processing.
- We design an agent workflow to implement this mechanism and propose an end-to-end training approach using the multi-conversation DAPO algorithm.

现有的长上下文任务处理方法主要有三种。第一种涉及通过移动位置嵌入来延长模型的上下文窗口的长度外推方法（Su 等人，2024; bloc97, 2023; Chen 等人，2023; Peng 等人，2023b; An 等人，2024），加上持续的预训练（Liu 等人，2023; Xiong 等人，2023; Gao 等人，2025）。尽管这些方法潜力巨大，但由于 $O(n^2)$ 计算复杂度，在应用于极长文本时，它们通常会导致性能下降和处理速度变慢。第二种方法利用稀疏注意力（Beltagy 等人，2020; Zhao 等人，2019; Xiao 等人，2023）和线性注意力机制（Child 等人，2019; Katharopoulos 等人，2020）来降低注意力的复杂度，从而更有效地处理更长的序列。然而，这通常需要从头开始训练，存在固有弊端，例如线性注意力在并行训练中面临困难，或稀疏注意力依赖于人类定义的模式。最后一类研究探索上下文压缩（Jiang 等人，2023; Li 等人，2023; Behrouz 等人，2024; Zhang 等人，2024），旨在将信息压缩到 token 级别或外部内存插件模块中。这类方法通常难以外推，并且需要集成额外的模块或上下文操作，这不可避免地会扰乱标准生成过程，并阻碍兼容性和并行化。因此，一个具有强大长上下文能力的成功大语言模型需要三位一体的支持：1) 处理无限长文本；2) 无显著性能下降的扩展；以及3) 具有线性复杂度的高效解码。为了追求这一目标，我们回归长上下文建模的基本直觉（Miller 等人，1956; Hochreiter & Schmidhuber, 1997; Graves 等人，2014; Weston 等人，2014）。当人类处理长上下文信息时，我们倾向于提取主要揭示的概念来把握全文的精髓，通常通过记录关键细节或使用速记法记录要点，同时丢弃冗余和不相关的数据。我们不会试图记住每一个事实或每一小段信息；相反，我们将智力精力集中在任务本身更重要的方面。这种选择性注意力不仅简化了过程，还有助于更高效地应对复杂问题。

遵循这种以人为本的直觉，我们提出了一种将强化学习（RL）用于为 LLMs 配备动态更新的固定长度“内存”的新方法，如图2所示。在推理过程中，LLM 逐段处理输入文本。当它读取每个段落时，模型会主动选择性地更新内存，然后所有相关消息在内存中聚合和协同后，该内存会为最终输出的生成做出贡献。这种巧妙的机制使 LLM 能够灵活处理任意文本长度，同时也在处理过程中保持线性时间复杂度，因为内存的长度是固定的，这导致模型的上下文窗口大小是固定的。这种基于段落的处理方法从单个长文本输入中生成多个输出，需要多次内存更新，以及最终一轮用于生成最终响应。训练这种能够实现跨多个独立上下文对话的代理工作流程，在当前 LLM 研究中仍是一个尚未探索的领域。现有系统通常通过交替工具调用或环境反馈来处理工作流轨迹，通过简单地将（Ouyang 等人，2025; Jin 等人，2025）它们或使用滑动窗口（Feng 等人，2025）方法，这在实践中缺乏灵活性和可扩展性。相反，我们的 MEMAGENT 方法提出将每个上下文无关对话视为一个优化目标。基于 DAPO（Yu 等人，2025）算法，我们实现了多卷积 DAPO，通过可验证结果奖励来优化任意代理工作流程。

在我们的实验中，一个经过 RL 训练的模型，其上下文窗口为适度的 8K（包含 1024 个 token 的内存和 5000 个 token 的文档片段），在训练了 60K 长度的文档后，在高达 350 万个 token 的文档上的问答（QA）任务中始终展现出卓越的能力，且性能不会下降，计算成本呈线性增长。这充分展示了我们长上下文内存方法的效率和可扩展性。

我们的主要贡献有三点：

- 我们提出了一种新方法，使大语言模型（LLMs）能够在有限的上下文窗口内，以线性时间复杂度处理任意长度的输入，从而克服了长上下文处理中的一个重大瓶颈。
- 我们设计了一种代理工作流程来实现该机制，并提出了一种使用多对话 DAPO 算法的端到端训练方法。

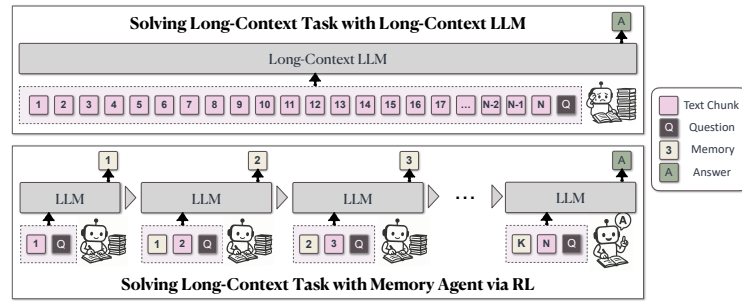


Figure 2: MEMAGENT is inspired by the way humans process long documents. It divides the document into multiple chunks and allows LLMs to process them iteratively, recording relevant information in memory. Finally, LLMs generate answers based on the information stored in the memory.

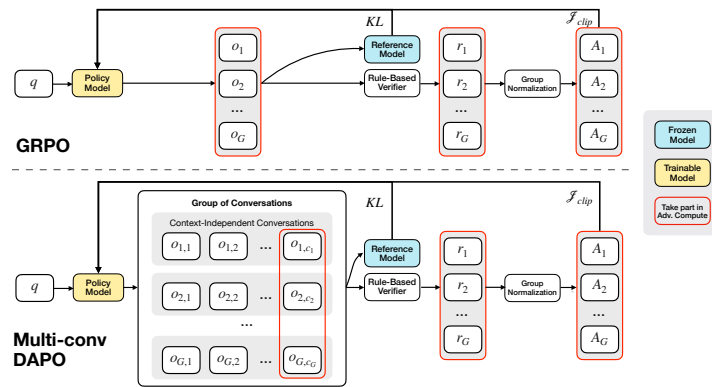


Figure 3: Comparison between vanilla GRPO and Multi-Conv DAPO. During the rollout phase of Multi-conv DAPO, each sample generates multiple conversations. The answer contained in the final conversation is used to compute the reward and advantage, which are then employed to optimize all preceding conversations.

- We empirically demonstrate that our RL-trained method allows models to extrapolate to vastly long documents with minimal performance degradation, pushing the boundaries of what is currently achievable in long-context LLM systems.

2 METHODOLOGY

In this section, we describe the details of MEMAGENT approach for solving long-context tasks, including the overall workflow (§ 2.1), Multi-conv RL algorithm for training MEMAGENT (§ 2.2) and the formal modeling of our architecture (§ 2.3).

2.1 THE MEMAGENT WORKFLOW: RL-SHAPED MEMORY FOR UNBOUNDED CONTEXTS

As illustrated in Figure 2, MEMAGENT views an arbitrarily long document not as a monolithic block but as a controlled *stream* of evidence. At every step, the model sees exactly two things: the next chunk of text and a compact, fixed-length *memory* that summarizes everything deemed important so far. Crucially, the memory is just a sequence of ordinary tokens inside the context window, so the core generation process of the base LLM remains unchanged.

After reading a new chunk, the model overwrites the previous memory with an updated one. This **overwrite** strategy seems almost too simple, yet it is precisely what enables the system to scale: because memory length never grows, the total compute per chunk stays $O(1)$ and end-to-end complexity is strictly linear to the number of chunks. We formulate the overwrite decision as a reinforcement

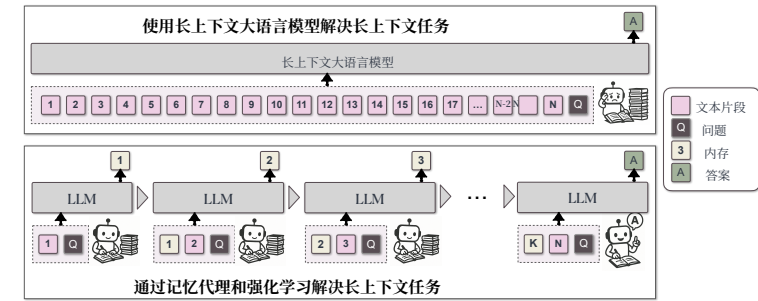


图2: MEMAGENT 受人类处理长文档方式启发。它将文档分割成多个片段，并允许大语言模型迭代处理这些片段，将相关信息记录在内存中。最后，大语言模型根据内存中存储的信息生成答案。

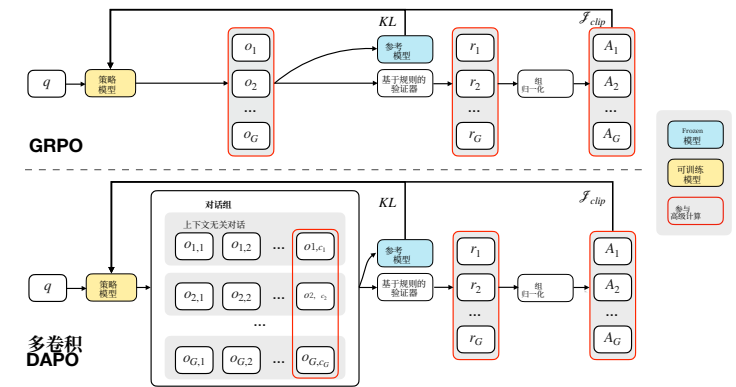


图3: vanilla GRPO 与 Multi-Conv DAPO 的对比。在 Multi-convDAPO 的展开阶段，每个样本会生成多个对话。最终对话中包含的答案用于计算奖励和优势，这些值随后被用于优化所有先前的对话。

- 我们通过实证证明，我们的RL训练方法使模型能够对极长的文档进行外推，且性能退化极小，从而将长上下文大语言模型系统的当前可达成边界推向了新的高度。

2 方法论

在本节中，我们描述了 MEMAGENT 用于解决长上下文任务的§ 2.1方法EMAGENT (§ 2.2) 以及我们架构的正式建模§ 2.3。

2.1 THEMEMAGENT 工作流程：RL 形状的内存在于无界上下文

s

如<样式 id='1'>图2</样式>所示，MEMAGENT将任意长度的文档视为一个受控的<样式 id='3'>证据流</样式>，而不是一个整体块。在每个步骤中，模型看到的是两件事：下一个文本片段和一个紧凑的固定长度<样式 id='5'>内存</样式>，该内存总结了迄今为止被认为重要的一切。关键在于，内存只是上下文窗口中的一系列普通token，因此基础LLM的核心生成过程保持不变。

读取新片段后，模型会用更新后的内存覆盖之前的内存。这种覆盖策略看似过于简单，但它正是系统实现可扩展性的关键：由于内存长度不会增长，每个片段的总体计算量保持 $O(1)$ 不变，端到端复杂度严格与片段数量线性相关。我们将覆盖决策表述为一个强化学习问题：代理在保留后来证明有用的信息以及丢弃会浪费宝贵 token 的干扰信息时获得奖励。通过使用我们新引入的多对话 DAPO 算法（详见 0.2）优化这一目标，模型学会在保留答案关键事实的同时进行激进压缩。

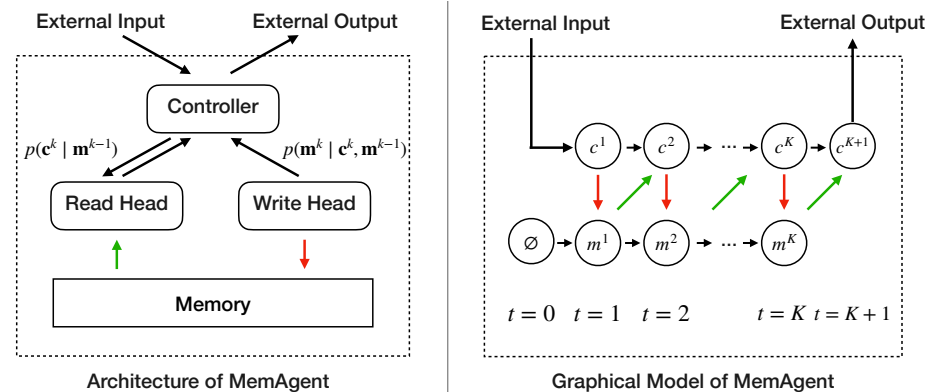


Figure 4: The architecture and graphic model of MEMAGENT. The memory is modeled as a latent memory variable, thereby enabling the decomposition of the autoregressive language model into multiple steps of reading from and writing to the memory.

learning problem: the agent is rewarded for retaining information that will later prove useful and for discarding distractors that would waste precious tokens. By optimizing this objective with our newly introduced multi-conversation DAPO algorithm (detailed in § 2.2), the model learns to compress aggressively while preserving answer-critical facts.

The workflow naturally decomposes inference into two modules. Within the **Context-Processing** module the model iterates over chunks, updating memory with a prompt template (Table 5, top). Once the stream is exhausted, a final **Answer-Generation** module is invoked (Table 5, bottom) where the model consults only the problem statement and the memory to produce its boxed answer. Because positional embeddings are never re-scaled or patched, the same tokenization and attention layout apply in both modules, unlocking the model’s latent length-extrapolation capability without any architectural modifications.

MEMAGENT therefore enjoys three benefits from this design: (1) **Unlimited length**: the document can be millions of tokens because it is processed as a stream; (2) **No performance cliff**: RL encourages the memory to retain exactly the information needed, yielding near-lossless extrapolation (Figure 1); (3) **Linear cost**: a constant window size implies decoding time and memory consumption grow linearly with input length ($O(N)$) (detailed in § B). This renders a practical recipe for turning any moderately context-sized LLM into an efficient long-context reasoner with minimal engineering overhead.

2.2 TRAINING MEMAGENT WITH MULTI-CONV RL

By viewing memory update in context processing for answer-generation tasks as part of the policy to be optimized by RL, we adopt the RLVR recipe (OpenAI, 2024; Guo et al., 2025; Seed et al., 2025) to train MEMAGENT. We adopt DAPO (Yu et al., 2025), an efficient and effective algorithm refined from Group Relative Policy Optimization (GRPO) (Shao et al., 2024), as our training algorithm. Due to the nature of our MEMAGENT approach, which generates multiple context-independent conversations for a single query as illustrated in Figure 2, we treat each conversation as an independent optimization target. This approach necessitates an extension of the loss computation from the conventional (group, token) structure to a new (group, conversation, token) dimensionality, as shown in Figure 3.

Formally, the policy model $\pi_{\theta_{\text{old}}}$ samples a group of G individual responses $\{o_{i,j}\}_{j=1}^G$ for an input x . Let n_i denote the number of generated conversations $(o_{i,1}, o_{i,2}, \dots, o_{i,n_i})$ for a given sample (q_i, a_i) . $o_{i,j}$ further decomposes into token-level outputs $(o_{i,j,1}, o_{i,j,2}, \dots, o_{i,j,|o_{i,j}|})$. The advantage value is derived from the conversation that contains the final answer, then uniformly applied to all conversations originating from the same sample, as shown in Eq 1. Following Dr. GRPO (Liu et al.,

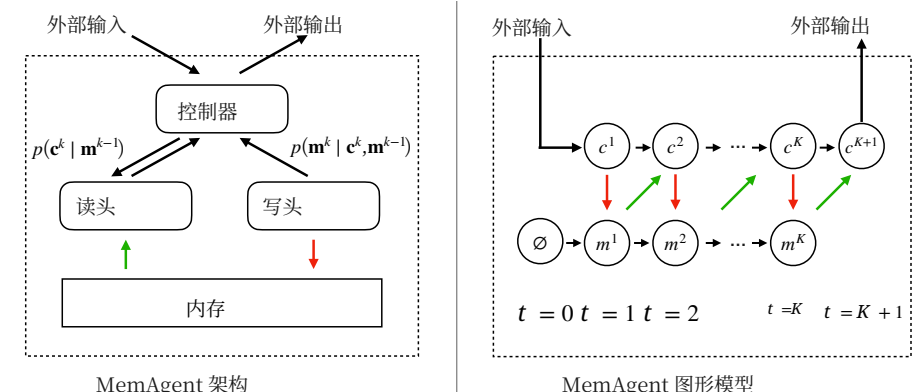


图4: MEMAGENT 的架构和图形模型。内存被建模为一个潜在记忆变量，从而能够将自回归语言模型分解为多个从内存中读取和写入的步骤。

学习问题：代理在保留后来证明有用的信息以及丢弃会浪费宝贵 token 的干扰信息时获得奖励。通过使用我们新引入的多对话 DAPO 算法（详见 § 2.2）优化这一目标，模型学会在保留答案关键事实的同时进行激进压缩。

工作流程自然地将推理分解为两个模块。在 **上下文处理** 模块中，模型遍历片段，使用提示模板（表 5，顶部）更新内存。一旦流用尽，就会调用最终的 **答案生成** 模块（表 5，底部），其中模型仅参考问题陈述和内存来生成其带框答案。由于位置嵌入从未被重新缩放或修补，两个模块中相同的分词和注意力布局适用，无需任何架构修改即可解锁模型的潜在长度外推能力。

MEMAGENT 因此，该设计为 MEMAGENT 带来了三大优势：(1)**无限长度**：文档可以包含数百万个 token，因为它被作为流处理；(2) **无性能悬崖**：强化学习鼓励内存保留所需信息，实现近乎无损的外推（图 1）；(3) **线性成本**：固定的窗口大小意味着解码时间和内存消耗随输入长度线性增长 ($O(N)$)（详见 § B。）这使得将任何中等上下文大小的 LLM 转换为高效的长期上下文推理器成为可能，且工程开销极小。

2.2 使用多卷积强化学习训练 MEMAGENT

通过将答案生成任务中上下文处理中的内存更新视为强化学习要优化的策略的一部分，我们采用了强化学习与视觉推理 (RLVR) 配方 (OpenAI, 2024; 郭等, 2025; Seed 等, 2025) 来训练 MEMAGENT。我们采用 DAPO (余等, 2025)，这是一种从组相对策略优化 (GRPO) (邵等, 2024) 改进而来的高效且有效的算法，作为我们的训练算法。由于我们的 MEMAGENT 方法的特点是针对单个查询生成多个上下文无关对话，如图 2 所示，我们将每个对话视为一个独立的优化目标。这种方法需要将损失计算从传统的(组, token) 结构扩展到新的(组, 对话, token) 维度，如图 3 所示。

形式上，策略模型 $\pi_{\theta_{\text{old}}}$ 从一组 G 个个体响应 $\{o_{i,j}\}_{j=1}^G$ 中为输入 x 进行采样。令 n_i 表示给定样本 (q_i, a_i) 生成的对话数量 $(o_{i,1}, o_{i,2}, \dots, o_{i,n_i})$ 。 $o_{i,j}$ 进一步分解为 token 级别的输出 $(o_{i,j,1}, o_{i,j,2}, \dots, o_{i,j,|o_{i,j}|})$ 。优势值是从包含最终答案的对话中得出的，然后均匀地应用于所有来自同一样本的对话，如 Eq 1 所示。遵循 GRPO 博士 (Liu 等人,

Table 1: Main experimental results comparing model performance across various context lengths. All values represent accuracy (%).

Model	Length									
	7K	14K	28K	56K	112K	224K	448K	896K	1.75M	3.5M
QwenLong-L1-32B	72.66	75.00	72.66	60.94	31.25	17.19	13.28	11.72	N/A	N/A
Qwen2.5-Instruct-14B-1M	60.16	60.94	50.00	57.03	50.00	37.50	8.59	0.00	N/A	N/A
Qwen2.5-Instruct-7B-1M	61.72	56.25	53.91	55.47	51.56	33.59	12.50	0.00	N/A	N/A
DS-Distill-Qwen-32B	70.31	66.41	65.62	46.88	23.44	13.28	7.81	7.03	N/A	N/A
DS-Distill-Qwen-14B	64.06	64.84	57.03	40.62	14.84	8.59	3.12	6.25	N/A	N/A
DS-Distill-Qwen-7B	30.47	12.50	3.12	0.00	0.00	0.78	0.00	0.00	N/A	N/A
Qwen2.5-Instruct-32B	69.53	64.84	60.16	51.56	44.53	21.88	14.06	7.03	N/A	N/A
Qwen2.5-Instruct-14B	75.00	67.19	57.03	54.69	44.53	21.88	10.94	2.34	N/A	N/A
Qwen2.5-Instruct-7B	52.34	57.03	51.56	44.53	32.81	13.28	6.25	1.56	N/A	N/A
RL-MEMAGENT-14B	80.47	82.03	82.03	83.59	81.25	77.34	79.69	75.78	78.91	71.09
RL-MEMAGENT-7B	81.25	81.25	82.03	80.47	79.69	75.78	76.56	74.22	77.34	71.88

2025), we do not devide the advantage by the standard deviation. Eq 2 describes our loss function.

$$\hat{A}_{i,j,t} = R_i - \text{mean}(\{R_i\}_{i=1}^G) \quad (1)$$

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_{i,j}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q, o_{i,j-1})} \left[\frac{1}{\sum_{i=1}^G \sum_{j=1}^{n_i} |o_{i,j}|} \sum_{i=1}^G \sum_{j=1}^{n_i} \sum_{t=1}^{|o_{i,j}|} \left(C_{i,j,t} - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) \right) \right] \quad (2)$$

$$\text{where } C_{i,j,t} = \min \left(r_{i,j,t}(\theta) \hat{A}_{i,j,t}, \text{clip} \left(r_{i,j,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}} \right) \hat{A}_{i,j,t} \right)$$

$$r_{i,j,t}(\theta) = \frac{\pi_{\theta}(o_{i,j,t} | q, o_{i,j,<t})}{\pi_{\theta_{\text{old}}}(o_{i,j,t} | q, o_{i,j,<t})}.$$

Following the RLVR recipe (Guo et al., 2025; Jin et al., 2025; Yu et al., 2025), we train the model with a final outcome reward computed by a rule-based verifier:

$$R(\hat{y}, y) = \mathbf{1}_{\text{is-equiv}(\hat{y}, y)} \quad (3)$$

where $\hat{y}$ is the predicted answer while y refers to the ground truth.

2.3 RETHINKING MEMAGENT FROM AUTOREGRESSIVE MODELING PERSPECTIVES

To get a deeper sense of the MEMAGENT design, we propose to re-think language-model factorization in the following fashion. A standard autoregressive LLM factorizes the joint likelihood of a sequence $\mathbf{x}_{1:N}$ as $p(\mathbf{x}_{1:N}) = \prod_{n=1}^N p(x_n | \mathbf{x}_{1:n-1})$, implicitly assuming that every past token (or at least its hidden state) must stay in the active context. This is what turns quadratic attention into the long-context bottleneck.

MEMAGENT replaces the unbounded history with a fixed-length *memory* $\mathbf{m} \in \mathbb{V}^M$, as shown in Figure 4. The input text is streamed through the model in K contiguous chunks $\mathbf{c}^1, \dots, \mathbf{c}^K$ (each of length $\leq C$). After chunk k is read, the model overwrites the panel with a new vector $\mathbf{m}^k$ that summarizes *all* evidence seen so far. Because $|\mathbf{m}^k| = M$ is constant, both compute and memory per step are $O(C + M)$, yielding an overall linear complexity $O(N)$.

Introducing the latent sequence $\mathbf{m}^{1:K-1}$ decomposes the original likelihood as

$$p(\mathbf{x}_{1:N}) = \sum_{\mathbf{m}^{1:K-1}} \prod_{k=1}^K \underbrace{p(\mathbf{c}^k | \mathbf{m}^{k-1})}_{\text{read}} \underbrace{p(\mathbf{m}^k | \mathbf{c}^k, \mathbf{m}^{k-1})}_{\text{write}}, \quad (4)$$

with base case $\mathbf{m}^0 = \emptyset$. Inside each chunk, we still run an ordinary transformer decoder, but conditioned on a *constant* context window $(\mathbf{c}^k, \mathbf{m}^{k-1})$. The read path factorizes token-by-token, $p(\mathbf{c}^k | \mathbf{m}^{k-1}) = \prod_{i=(k-1)C+1}^{kC} p(x_i | \mathbf{x}_{1:i-1}, \mathbf{m}^{k-1})$, while the write path generates the next memory in the same autoregressive fashion.

表1: 主要实验结果, 比较了不同上下文长度下的模型性能。所有数值表示准确率 (%)。

模型	长度									
	7K	14K	28K	56K	112K	224K	448K	896K	1.75M	3.5M
QwenLong-L1-32B	72.66	75.00	72.66	60.94	31.25	17.19	13.28	11.72	N/A	N/A
Qwen2.5-Instruct-14B-1M	60.16	60.94	50.00	57.03	50.00	37.50	8.59	0.00	N/A	N/A
Qwen2.5-Instruct-7B-1M	61.72	56.25	53.91	55.47	51.56	33.59	12.50	0.00	N/A	N/A
DS-Distill-Qwen-32B	70.31	66.41	65.62	46.88	23.44	13.28	7.81	7.03	N/A	N/A
DS-Distill-Qwen-14B	64.06	64.84	57.03	40.62	14.84	8.59	3.12	6.25	N/A	N/A
DS-Distill-Qwen-7B	30.47	12.50	3.12	0.00	0.00	0.78	0.00	0.00	N/A	N/A
Qwen2.5-Instruct-32B	69.53	64.84	60.16	51.56	44.53	21.88	14.06	7.03	N/A	N/A
Qwen2.5-Instruct-14B	75.00	67.19	57.03	54.69	44.53	21.88	10.94	2.34	N/A	N/A
Qwen2.5-Instruct-7B	52.34	57.03	51.56	44.53	32.81	13.28	6.25	1.56	N/A	N/A
RL-MEMAGENT-14B	80.47	82.03	82.03	83.59	81.25	77.34	79.69	75.78	78.91	71.09
RL-MEMAGENT-7B	81.25	81.25	82.03	80.47	79.69	75.78	76.56	74.22	77.34	71.88

2025), 我们不将优势值除以标准差。Eq 2 描述了我们的损失函数

$$\hat{A}_{i,j,t} = R_i - \text{mean}(\{R_i\}_{i=1}^G) \quad (1)$$

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_{i,j}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q, o_{i,j-1})} \left[\frac{1}{\sum_{i=1}^G \sum_{j=1}^{n_i} |o_{i,j}|} \sum_{i=1}^G \sum_{j=1}^{n_i} \sum_{t=1}^{|o_{i,j}|} \left(C_{i,j,t} - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) \right) \right] \quad (2)$$

$$\text{where } C_{i,j,t} = \min \left(r_{i,j,t}(\theta) \hat{A}_{i,j,t}, \text{clip} \left(r_{i,j,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}} \right) \hat{A}_{i,j,t} \right)$$

$$r_{i,j,t}(\theta) = \frac{\pi_{\theta}(o_{i,j,t} | q, o_{i,j,<t})}{\pi_{\theta_{\text{old}}}(o_{i,j,t} | q, o_{i,j,<t})}.$$

遵循RLVR配方 (Guo等人., 2025; Jin等人., 2025; Yu等人., 2025), 我们使用基于规则的验证器计算最终结果奖励来训练模型:

$$R(\hat{y}, y) = \mathbf{1}_{\text{is-equiv}(\hat{y}, y)} \quad (3)$$

其中 $\hat{y}$ 是预测答案, 而 y 是指真实值。

2.3 重新思考 MEMAGENT 从自回归建模视角出发

为了更深入地理解 MEMAGENT 的设计, 我们建议以以下方式重新思考语言模型的分解。标准的自回归大语言模型将序列 $\mathbf{x}_{1:N}$ 的联合概率分解为 $p(\mathbf{x}_{1:N}) = \prod_{n=1}^N p(x_n | \mathbf{x}_{1:n-1})$, 隐含地假设每个过去的 token (或者至少是其隐藏状态) 必须保持在活跃的上下文中。这就是为什么二次注意力会变成成长上下文的瓶颈。

MEMAGENT 用固定长度的内存 $\mathbf{m} \in \mathbb{V}^M$ 替代无界历史, 如图 4 所示。输入文本以 K 连续的片段 $\mathbf{c}^1, \dots, \mathbf{c}^K$ (每个长度为 $\leq C$) 流经模型。在读取片段 k 后, 模型用一个新的向量 $\mathbf{m}^k$ 覆盖面板, 该向量总结了迄今为止看到的所有证据。因为 $|\mathbf{m}^k| = M$ 是恒定的, 每步的 compute 和内存都是 $O(C + M)$, 从而产生整体线性复杂度 $O(N)$ 。

介绍潜在序列 $\mathbf{m}^{1:K-1}$ 将原始似然分解为

$$p(\mathbf{x}_{1:N}) = \sum_{\mathbf{m}^{1:K-1}} \prod_{k=1}^K \underbrace{p(\mathbf{c}^k | \mathbf{m}^{k-1})}_{\text{read}} \underbrace{p(\mathbf{m}^k | \mathbf{c}^k, \mathbf{m}^{k-1})}_{\text{write}}, \quad (4)$$

以基本情况 $\mathbf{m}^0 = \emptyset$ 为基础。在每个片段中, 我们仍然运行一个普通的 transformer 解码器, 但以一个固定上下文窗口 $(\mathbf{c}^k, \mathbf{m}^{k-1})$ 为条件。读取路径按 token 分解, $p(\mathbf{c}^k | \mathbf{m}^{k-1}) = \prod_{i=(k-1)C+1}^{kC} p(x_i | \mathbf{x}_{1:i-1}, \mathbf{m}^{k-1})$, 而写入路径则以同样的自回归方式生成下一个内存。

Table 2: Model performance on LongBench-SUM. All values represent recall rates (%). **Bold** marks the highest value and Underline marks the second highest value in each column.

Model	GOV REPORT				QMSUM			
	ROUGE-1	ROUGE-2	ROUGE-L	AVG	ROUGE-1	ROUGE-2	ROUGE-L	AVG
Qwen2.5-Instruct-32B	23.67	8.46	12.57	14.90	47.77	11.29	28.17	29.08
Qwen2.5-Instruct-14B	31.19	10.96	14.96	19.04	47.53	11.46	28.28	29.09
Qwen2.5-Instruct-7B	30.91	11.68	15.20	19.26	46.64	12.01	28.33	28.99
QwenLong-L1	27.60	8.20	13.07	16.29	39.44	8.24	23.56	23.74
Qwen2.5-Instruct-14B-1M	30.58	11.93	15.51	<u>19.34</u>	47.31	13.13	29.07	29.84
Qwen2.5-Instruct-7B-1M	31.02	11.47	15.30	19.26	46.72	12.33	28.66	29.24
DS-Distill-Qwen-32B	26.13	8.86	12.98	15.99	39.09	8.75	23.96	23.93
DS-Distill-Qwen-14B	28.24	9.72	13.78	17.25	41.25	8.95	25.00	25.07
DS-Distill-Qwen-7B	<u>33.30</u>	9.39	14.59	19.10	34.33	5.97	21.57	20.62
RL-MEMAGENT-14B	37.16	<u>12.03</u>	16.23	21.80	50.21	<u>12.70</u>	31.27	31.39
RL-MEMAGENT-7B	30.28	12.37	<u>15.37</u>	<u>19.34</u>	48.49	14.41	<u>30.91</u>	<u>31.27</u>

In our formulation, the model’s reading and writing operations over the context constitute an Markov Decision Process(MDP) and the objective of RL is to optimize the final reward obtained by this MDP. Therefore, MemAgent’s learning objective is to generate a read–write memory trajectory that maximizes the reward, which corresponds to learning an optimal distribution over memory states conditioned on the input context. This further theoretically illustrates the intrinsic unity between our RL formulation and long-text modeling.

3 EXPERIMENTS

3.1 EXPERIMENTAL SETUP

Training Details. To maintain comparability with previous work, we choose Qwen2.5-7B-Instruct and Qwen2.5-14B-Instruct (Yang et al., 2024) as backbone models. We implement the framework for multi-conversation with independent contexts based on verl (Sheng et al., 2024). We employ a two-stage curriculum RL strategy:

Table 3: Model performance on LongBench-QA.

Method	2Wiki	HQA	MuSiQue	NQA	Qasper	AVG
QwenLong-L1-32B	83.0	69.5	51.0	26.0	24.0	50.7
Qwen2.5-Instruct-14B-1M	70.5	65.5	35.0	22.0	22.0	43.0
Qwen2.5-Instruct-7B-1M	67.5	58.5	26.0	21.5	24.0	39.5
DS-Distill-Qwen-32B	83.5	69.0	47.5	24.0	21.0	49.0
DS-Distill-Qwen-14B	83.5	67.0	42.0	22.0	22.0	47.3
DS-Distill-Qwen-7B	47.0	31.5	7.5	4.0	17.5	21.5
Qwen2.5-Instruct-32B	68.5	66.0	37.0	24.5	22.5	43.7
Qwen2.5-Instruct-14B	67.0	63.5	39.0	20.0	20.5	42.0
Qwen2.5-Instruct-7B	52.5	59.0	24.0	19.0	19.0	34.7
MEMAGENT-14B	79.0	73.0	52.0	25.0	26.0	51.0
MEMAGENT-7B	74.0	69.5	47.0	21.5	29.0	48.2

1) stage I focuses on enabling the model to acquire fundamental memory capabilities; 2) stage II trains the model to transfer these capabilities to more diverse contexts and challenging tasks. Specific hyperparameters for training are detailed in § A.3.

During training, we intentionally limit the model to an 8K context window to demonstrate its extrapolation capabilities. This 8K window is allocated as follows: 1024 tokens for the query, 5000 tokens for the context chunk, 1024 tokens for the memory, and 1024 tokens for the output, with the remaining tokens reserved for the chat template.

Benchmarks. We conduct comprehensive evaluations on several long-text benchmarks to assess the model’s capabilities across various text types and tasks.

1. **RULER-HQA.** This benchmark is created using the same synthetic method as in the first-stage training data. It consists of tasks with a moderate information density and controllable length, where the context distribution is close to natural language, serving as a quantitative evaluation of extrapolation performance.
2. **LongBench-QA.** This benchmark is composed of NarrativeQA (Kočískỳ et al., 2018), Qasper (Dasigi et al., 2021), HotpotQA (Yang et al., 2018), 2WikiMultihopQA Ho et al. (2020), and MuSiQue (Trivedi et al., 2022). The tasks are relatively short but have a high information density, which severely tests the model’s flexible memory management. It also evaluates the model’s ability to generalize its memory capabilities to various materials, such as novels, news articles, and Wiki items.

表2：模型在LongBench-SUM上的性能。所有值表示召回率（%）。**加粗**标记每列中最高值，下划线标记每列中第二高的值。_____

模型	GOV REPORT				QMSUM			
	ROUGE-1	ROUGE-2	ROUGE-L	AVG	ROUGE-1	ROUGE-2	ROUGE-L	AVG
Qwen2.5-Instruct-32B	23.67	8.46	12.57	14.90	47.77	11.29	28.17	29.08
Qwen2.5-Instruct-14B	31.19	10.96	14.96	19.04	47.53	11.46	28.28	29.09
Qwen2.5-Instruct-7B	30.91	11.68	15.20	19.26	46.64	12.01	28.33	28.99
QwenLong-L1	27.60	8.20	13.07	16.29	39.44	8.24	23.56	23.74
Qwen2.5-Instruct-14B-1M	30.58	11.93	15.51	<u>19.34</u>	47.31	13.13	29.07	29.84
Qwen2.5-Instruct-7B-1M	31.02	11.47	15.30	19.26	46.72	12.33	28.66	29.24
DS-Distill-Qwen-32B	26.13	8.86	12.98	15.99	39.09	8.75	23.96	23.93
DS-Distill-Qwen-14B	28.24	9.72	13.78	17.25	41.25	8.95	25.00	25.07
DS-Distill-Qwen-7B	<u>33.30</u>	9.39	14.59	19.10	34.33	5.97	21.57	20.62
RL-MEMAGENT-14B	37.16	<u>12.03</u>	16.23	21.80	50.21	<u>12.70</u>	31.27	31.39
RL-MEMAGENT-7B	30.28	12.37	<u>15.37</u>	<u>19.34</u>	48.49	14.41	<u>30.91</u>	<u>31.27</u>

在我们的公式中，模型对上下文的读取和写入操作构成一个马尔可夫决策过程（MDP），而强化学习（RL）的目标是优化这个 MDP 获得的最终奖励。因此，MemAgent 的学习目标是为输入上下文生成一个读取-写入内存轨迹，以最大化奖励，这对应于学习一个在输入上下文条件下对内存状态的最优分布。这进一步从理论上说明了我们 RL 公式与长文本建模之间的内在统一性。

3 实验

3.1 实验设置

训练详情。 为了与前人工作保持可比性，我们选择 Qwen2.5-7B-Instruct 和 Qwen2.5-14B-Instruct (Yang 等人, 2024) 作为骨干模型。我们基于 verl (Sheng 等人,2024) 实现了多对话独立上下文的框架。

表 3：模型在 LongBench-QA 上的性能。

方法	2Wiki	HQA	MuSiQue	NQA	Qasper	AVG
QwenLong-L1-32B	83.0	69.5	51.0	26.0	24.0	50.7
Qwen2.5-Instruct-14B-1M	70.5	65.5	35.0	22.0	22.0	43.0
Qwen2.5-Instruct-7B-1M	67.5	58.5	26.0	21.5	24.0	39.5
DS-Distill-Qwen-32B	83.5	69.0	47.5	24.0	21.0	49.0
DS-Distill-Qwen-14B	83.5	67.0	42.0	22.0	22.0	47.3
DS-Distill-Qwen-7B	47.0	31.5	7.5	4.0	17.5	21.5
Qwen2.5-Instruct-32B	68.5	66.0	37.0	24.5	22.5	43.7
Qwen2.5-Instruct-14B	67.0	63.5	39.0	20.0	20.5	42.0
Qwen2.5-Instruct-7B	52.5	59.0	24.0	19.0	19.0	34.7
MEMAGENT-14B	79.0	73.0	52.0	25.0	26.0	51.0
MEMAGENT-7B	74.0	69.5	47.0	21.5	29.0	48.2

1) 第一阶段侧重于使模型具备基本的内存能力；2) 第二阶段训练模型将这些能力迁移到更多样化的上下文和更具挑战性的任务中。具体的训练超参数详见 § A.3。

在训练过程中，我们有意将模型的上下文窗口限制为8K，以展示其外推能力。这个8K窗口的分配如下：1024个token用于查询，5000个token用于上下文片段，1024个token用于内存，1024个token用于输出，剩余的token用于聊天模板。

基准测试。 我们在多个长文本基准测试上进行了全面评估，以评估模型在不同文本类型和任务中的能力。

1. **RULER-HQA.** 该基准测试使用与第一阶段训练数据相同的合成方法创建。它包含信息密度中等且长度可控的任务，其中上下文分布接近自然语言，作为外推性能的定量评估。
2. **LongBench-QA.** 这个基准测试由叙事问答（Ko`cisk`等, 2018），Qasper（Dasigi等, 2021），热点问答（Yang等, 2018），2维基多跳问答 Ho等(2020)，以及MuSiQue（Trivedi等, 2022）组成。这些任务相对较短，但信息密度很高，这严重考验了模型的灵活内存管理能力。它还评估了模型将内存能力泛化到各种材料（如小说、新闻文章和维基条目）的能力。

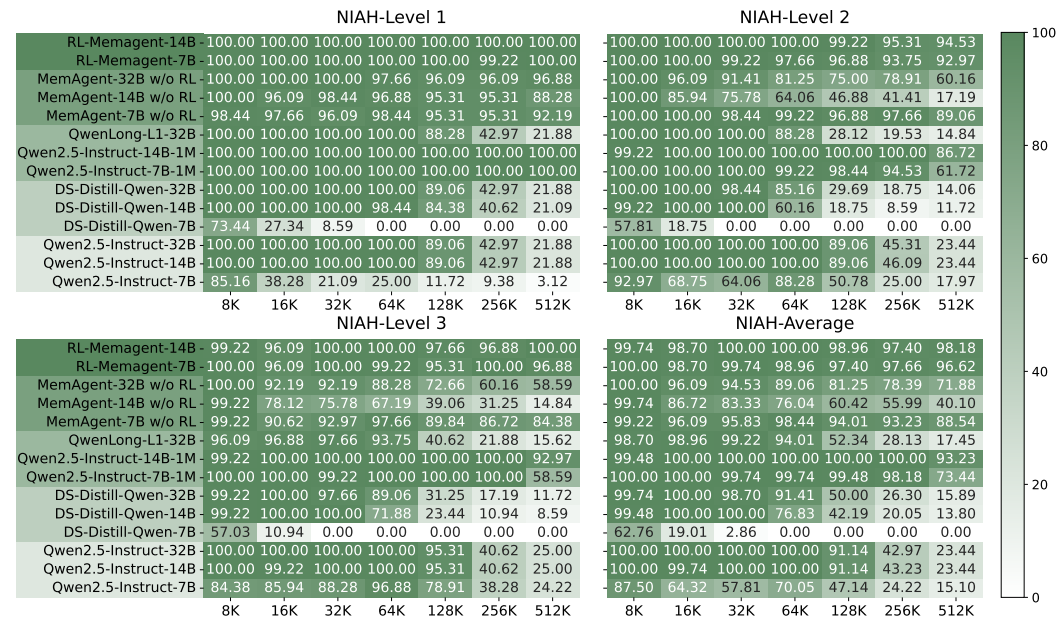


Figure 5: Performance heatmaps on NIAH benchmark across different context lengths.

3. **NIAH.** Needle in a haystack (NIAH) (Kamradt, 2023) is a series of extremely long synthetic tasks with very low information density. To succeed, the model must identify key information and maintain its integrity throughout a long process, thereby testing the robustness of memory.
4. **LongBench-SUM.** We also adopt two long-context summar tasks, GovReport(Huang et al., 2021) and QMSum(Zhong et al., 2021) from LongBench(Bai et al., 2024) to evaluate the performance in different task category that is different from retrieval QA.

Baselines. We use DeepSeek-R1-Distill-Qwen (Guo et al., 2025), Qwen-2.5-Instruct-1M (Yang et al., 2025), Qwen-2.5-Instruct (Yang et al., 2024) and QwenLong-L1 (Wan et al., 2025) as baselines. Their generation configurations are shown in Table 6, while MEMAGENT uses the same context management as described previously in **Training Details**. We also compare MEMAGENT with other agent method, detailed in § D.2.

3.2 MAIN RESULTS

RULER-HQA. The results are reported in Table 10. We conduct a comparative analysis of all model performances with context lengths ranging from 7K to 896K. For MEMAGENT, we extend the evaluation to ultra-long contexts of 1.75M and 3.5M to assess its extrapolation capabilities.

MEMAGENT exhibits remarkable length extrapolation capabilities with only marginal performance decay as the input context-length increases. In contrast, baseline models show distinct failure patterns. DS-Distill-Qwen series show rapid performance degradation. QwenLong-L1 maintains reasonable performance within its training length but experiences substantial degradation afterward. The Qwen2.5-Instruct-1M series maintains acceptable performance up to 112K tokens, but the performance deteriorates to zero at 896K tokens, well before reaching their theoretical 1M token capacity. This suggests that despite extended context windows, these models struggle with effective information utilization in ultra-long contexts.

LongBench-QA. The results on the LongBench-QA benchmark are presented in Table 3. MEMAGENT demonstrates superior overall performance, outperforming larger long-context or reasoning models. Reasoning models such as the DS-Distill families and the QwenLong model which are trained on a complex dataset, exhibit strong performance. In contrast, the Qwen2.5-Instruct-1M series shows limited improvement over its backbone model. This suggests that LongBench-QA emphasize

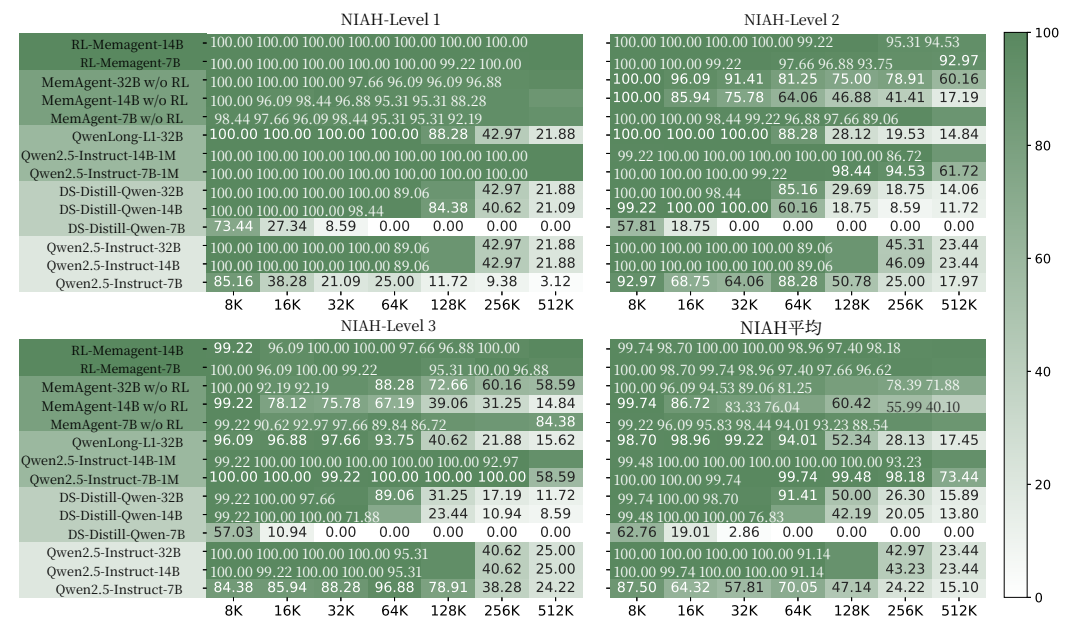


图5: 在不同上下文长度下, NIAH基准上的性能热图

3. **NIAH.** 针锋相对 (NIAH) (Kamradt, 2023) 是一系列极长的合成任务, 信息密度非常低。要成功完成这些任务, 模型必须识别关键信息, 并在整个长过程中保持其完整性, 从而考验内存的鲁棒性。
4. **LongBench-SUM.** 我们还采用两个长上下文摘要任务, GovReport(Huang等人。 , 2021)和QMSum(Zhong等人。 , 2021)来自LongBench(Bai等人。 , 2024), 以评估在不同任务类别 (不同于检索QA) 中的性能。

基线. 我们使用DeepSeek-R1-Distill-Qwen (Guo等人。 , 2025), Qwen-2.5-Instruct-1M (Yang等人。 , 2025), Qwen-2.5-Instruct (Yang等人。 , 2024)和QwenLong-L1 (Wan等人。 , 2025)作为基线。它们的生成配置显示在表6中, 而MEMAGENT使用与之前在训练细节中描述的不同上下文管理。我们还将MEMAGENT与其他代理方法进行比较, 详细信息见 § D.2。

3.2 主要结果

RULER-HQA. 结果报告在表10中。我们对所有模型性能进行了比较分析, 上下文长度范围从7K到896K。对于MEMAGENT, 我们将评估扩展到1.75M和3.5M的超长上下文, 以评估其外推能力。

MEMAGENT表现出卓越的长度外推能力, 随着输入上下文长度的增加, 性能衰减仅限于边缘。相比之下, 基线模型显示出明显的失败模式。DS-Distill-Qwen系列表现出快速的性能退化。QwenLong-L1在其训练长度内保持合理性能, 但之后经历大幅退化。Qwen2.5-Instruct-1M系列在112K token内保持可接受性能, 但在896K token时性能降至零, 远未达到其理论上的1M token容量。这表明尽管扩展了上下文窗口, 这些模型在超长上下文中仍难以有效利用信息。

LongBench-QA. LongBench-QA基准测试的结果展示在表3中。MEMAGENT表现出卓越的整体性能, 优于更大的长上下文或推理模型。在复杂数据集上训练的推理模型, 如DS-Distill系列和QwenLong模型, 展现出强劲的性能。相比之下, Qwen2.5-Instruct-1M系列在其主干模型上的改进有限。这表明LongBench-QA强调

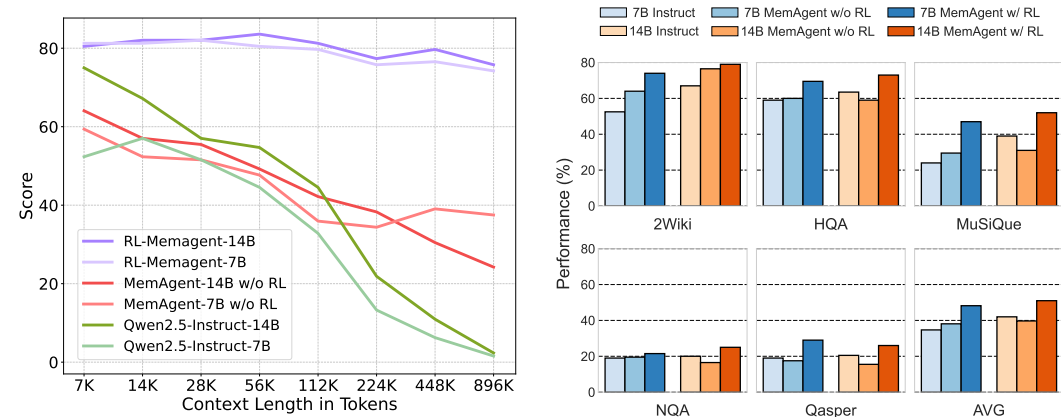


Figure 6: Ablation result of RL training on RULER-HQA.

Figure 7: Ablation result of RL training on Longbench-QA.

a deeper understanding of text rather than simple retrieval ability. The performance of MEMAGENT demonstrates that the memory capabilities acquired through reinforcement learning are generalizable.

NIAH. We adopt three variants of NIAH from the RULER benchmark [Hsieh et al. \(2024\)](#) with increasing difficulty across three levels. As depicted in Figure 5, the majority of baselines struggle to maintain consistent performance even within a 128K context window, even Qwen2.5-Instruct-1M also experience a performance drop at 512K. RL-MEMAGENT, despite suffering some performance fluctuations, shows only a minimal performance loss of less than 5% at 512K. This robust performance is particularly noteworthy given that the evaluation at 512K involves more than 100 turns of dialogue.

LongBench-SUM. We evaluate summary quality by the recall scores of $\text{ROUGE}-\{1, 2, L\}$. RL-MEMAGENT achieves SOTA on almost all metrics, demonstrating that the model has learned general memory and context management capabilities, rather than abilities specific to the QA task.

3.3 ABLATION STUDY

3.3.1 RL TRAINING

To investigate the impact of reinforcement learning, we conduct ablation experiments. The results of RULER-HQA and NIAH are presented in Figure 6 and Figure 5, respectively. MEMAGENT without reinforcement learning training outperforms the backbone models; however, it still exhibits a substantial decline in performance as the input length increases. The results of Longbench-QA, shown in Figure 7, demonstrate that directly applying MEMAGENT leads to only marginal or even negative improvements. In contrast, RL-MEMAGENT achieves significant improvements in both evaluation scenarios, indicating reinforcement learning training is essential to develop generalizable and robust memory abilities.

3.3.2 MEMORY LENGTH

Selecting an appropriate MEMAGENT setting involves certain trade-offs. A larger memory size allows the model to store more useful information, but it also introduces challenges in memory management and increases the likelihood of redundancy. Conversely situation may lead to insufficient storage capacity, leaving the model without the necessary references.

To achieve a reasonable compression ratio while keeping the total context length within 8,192 tokens, we set the default configuration of MEMAGENT to use a 1,024-token memory and context chunks of 5,000 tokens, based on preliminary validation results.

To investigate the effect of hyperparameter choices, we conduct an ablation study on memory length ranging from 256 to 4096. The results presented in Figure 8 and Figure 9, showing that our chosen configuration constitutes a reasonable sweet spot, and that MEMAGENT’s performance is robust over different memory size. We further examine the impact of varying the context size in § D.1 and observe similar trends.

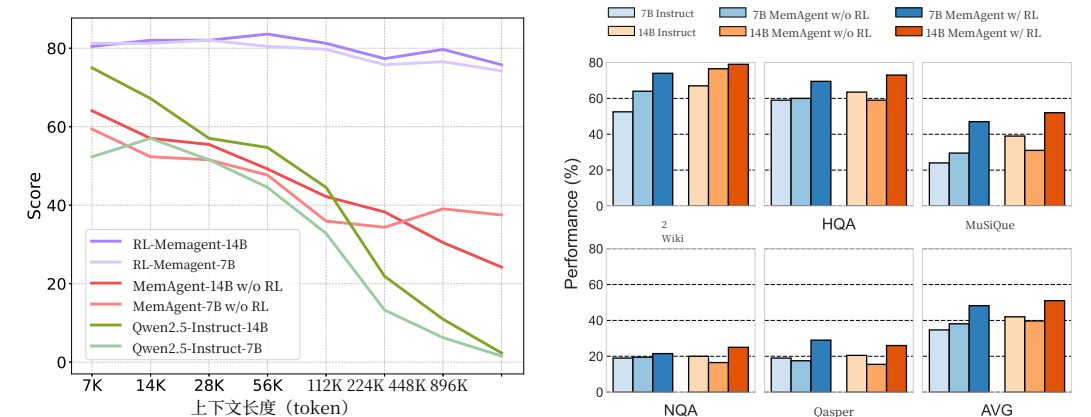


图6: RULER-HQA上强化学习训练的消融结果。图7: Longbench-QA上强化学习训练的消融结果。

对文本更深入的理解，而非简单的检索能力。MEMAGENT的性能表明，通过强化学习获得的内存能力是可泛化的。

NIAH. 我们从RULER基准中采用了三种NIAH变体[Hsieh等人\(2024\)](#)，难度逐级递增。如图5所示，大多数基线即使在128K上下文窗口内也难以保持稳定性能，即使Qwen2.5-Instruct-1M在512K时也出现了性能下降。RL-MEMAGENT尽管存在一些性能波动，但在512K时仅损失不到5%的性能。考虑到512K评估涉及超过100轮的对话，这种稳健性能尤为突出。

LongBench-SUM. 我们通过ROUGE- $\{1, 2, L\}$ 的召回分数评估摘要质量。RL-MEMAGENT在几乎所有指标上均达到SOTA，表明模型已学习到通用的记忆和上下文管理能力，而非特定于QA任务的能力。

3.3 消融研究

3.3.1 强化学习训练

为研究强化学习的影响，我们进行了消融实验。RULER-HQA 和 NIAH 的结果分别展示在图6和图5中。未进行强化学习训练的 MEMAGENT 表现优于骨干模型；然而，随着输入长度的增加，其性能仍出现显著下降。Longbench-QA 的结果（如图 7所示）表明，直接应用 MEMAGENT 仅带来微小的甚至负面的改进。相比之下，RL-MEMAGENT 在两种评估场景中均实现了显著提升，这表明强化学习训练对于开发可泛化且鲁棒的内存能力至关重要。

3.3.2 内存长度

选择合适的 MEMAGENT 设置涉及一定的权衡。更大的内存大小允许模型存储更多信息，但也引入了内存管理和冗余风险。反之，则可能导致存储容量不足，使模型缺乏必要的参考。

要在保持总上下文长度不超过8,192个token的同时实现合理的压缩率，我们根据初步验证结果，将MEMAGENT的默认配置设置为使用1,024个token的内存和5,000个token的上下文片段。

为了研究超参数选择的影响，我们对内存长度从 256 到 4096 进行了消融研究。图 8和图 9展示的结果表明，我们选择的配置构成了一个合理的甜点，并且 MEMAGENT 的性能在不同内存大小下都很稳健。我们进一步在 § D.1中检查了改变上下文大小的影响，并观察到类似的趋势。

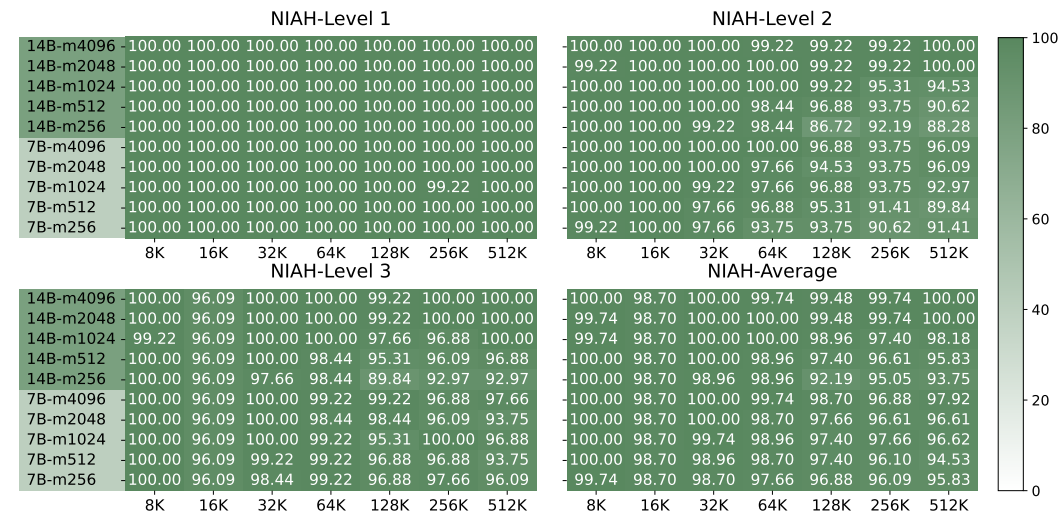


Figure 8: Ablation result of memory-length on NIAH

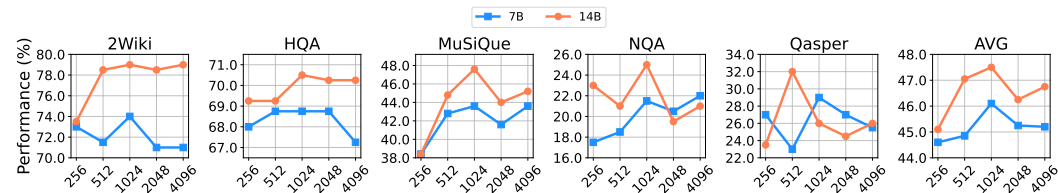


Figure 9: Ablation result of memory-length on Longbench

3.3.3 CONTEXT DISTRIBUTION

Although our experiments show that MEMAGENT can effectively extrapolate to a length of 3.5M tokens, we still wish to examine whether MEMAGENT is affected by issues such as information-overwritten and the lost-in-the-middle phenomenon. Our hypothesis is that overcoming such problem is a natural result of end-to-end optimization. During training, the model learns to preserve and track critical information in order to maximize the final reward.

To validate this hypothesis, we carefully design a set of probing experiments based on RULER-HQA, where the context is consist of some key information and many distractors. We divided the key information into two groups and placed them at different positions within the context. We constructed five settings: (0%, 100%), (20%, 80%), (40%, 60%), (0%, 20%), and (80%, 100%), where 0% indicates the beginning of the context and 100% means the end of the context.

For example, in the (0%, 100%) case, the model sees one piece of key information at the very beginning and the other only at the final memory update step. This represents one of the most challenging scenarios for the information-overwritten problem. While (40%, 60%) may serve as a challenging lost-in-the-middle setting.

The results shown in table 4 indicates that MemAgent remains consistently robust across all patterns without exhibiting any catastrophic performance degradation. This strongly supports our hypothesis that the general memory abilities acquired through trial and error are not tied to any particular context pattern.

4 RELATED WORK

Long Context LLMs. Extrapolation methods for RoPE-based LLMs (Su et al., 2024), such as NTK (bloc97, 2023), PI (Chen et al., 2023), YaRN (Peng et al., 2023b) and DCA (An et al., 2024),

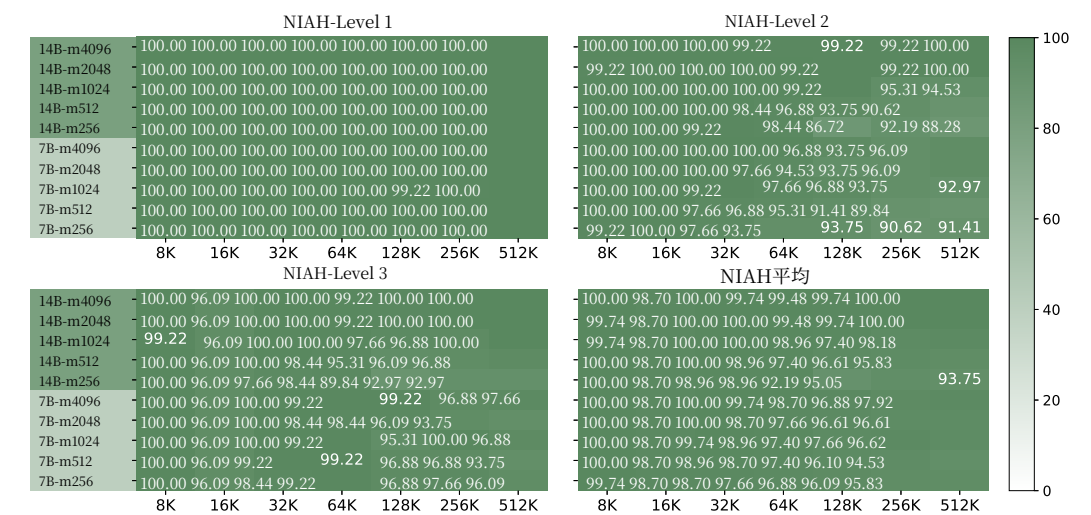


图8: NIAH的内存长度消融结果

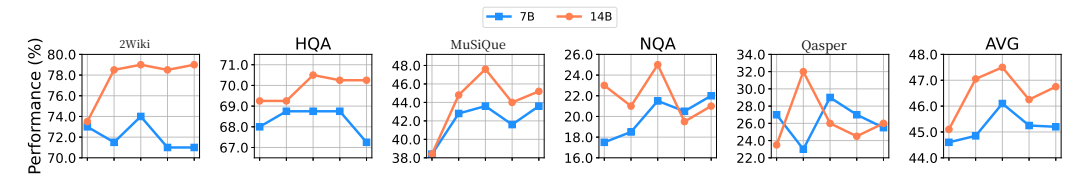


图9: Longbench的内存长度消融结果

3.3.3 上下文分布

尽管我们的实验表明 MEMAGENT 可以有效地外推到 3.5M 个 token 的长度，但我们仍然希望检验 MEMAGENT 是否会受到信息覆盖和信息丢失在中间现象等问题的影响。我们的假设是克服这些问题是端到端优化的自然结果。在训练过程中，模型学习保留和跟踪关键信息，以最大化最终奖励。

为验证这一假设，我们基于RULER-HQA精心设计了一套探测实验，其中上下文包含一些关键信息和许多干扰信息。我们将关键信息分为两组，并将其放置在上下文的不同位置。我们构建了五种设置：(0%, 100%)、(20%, 80%)、(40%, 60%)、(0%, 20%)和(80%, 100%)，其中0%表示上下文的开始，100%表示上下文的结束。

例如，在(0%, 100%)的情况下，模型在上下文最开始看到一件关键信息，另一件仅在最终内存更新步骤才看到。这代表了信息覆盖问题中最具挑战性的场景之一。而(40%, 60%)可能作为一个具有挑战性的中间丢失设置。

表 4 中的结果表明，MemAgent 在所有模式中都保持稳健，且未出现灾难性性能退化。这有力地支持了我们的假设：通过试错获得的通用记忆能力并非与任何特定上下文模式相关。

4 相关工作

长上下文大语言模型。 基于RoPE的大语言模型的插值方法 (Su等人, 2024)，例如NTK (bloc97, 2023)，PI (陈等人, 2023)，YaRN (彭等人, 2023b) 和 DCA (An 等人, 2024)，

Table 4: Probe experiment results. **Ctx. Dist.** denotes the context distribution, where the two numbers correspond to the relative positions of the two key-information groups within the entire context. 0% means the beginning and 100% means the end. **random** indicates randomly shuffling all context items, consistent with the setup in the main experiment. The other rows show the performance difference relative to **random**. All values represent accuracy (%).

Model	Ctx. Dist.	Length							
		7K	14K	28K	56K	112K	224K	448K	AVG
14B	random	80.47	82.03	82.03	83.59	81.25	77.34	79.69	75.78
	0% 20%	+3.91	-3.91	+3.13	+1.57	0.00	+4.69	+3.90	+1.90
	0% 100%	+3.12	+0.78	+3.13	-3.12	+1.56	+7.82	+6.25	+2.79
	20% 80%	+0.78	-3.12	+2.35	+0.79	-3.13	+5.47	-3.13	0.00
	40% 60%	+1.56	+2.35	-2.34	-3.12	-1.56	+3.13	-0.78	-0.11
	80% 100%	-2.35	0.00	+1.56	+0.79	+3.13	0.00	+1.56	+0.67
7B	random	81.25	81.25	82.03	80.47	79.69	75.78	76.56	79.58
	0% 20%	-1.56	-0.78	+3.13	+3.91	+3.12	+5.47	+3.13	+2.35
	0% 100%	+0.78	0.00	+2.35	+1.56	+3.90	+4.69	+2.35	+2.23
	20% 80%	-0.78	-0.78	+2.35	0.00	0.00	0.00	+3.13	+0.56
	40% 60%	0.00	+1.56	+3.13	+0.78	-3.91	+3.13	+5.47	+1.45
	80% 100%	+1.56	0.00	+0.78	-0.78	+0.78	+4.69	0.00	+1.00

modify the components of positional embeddings, enabling the model to capture long-range semantic dependencies. On the other hand, Linear attention mechanisms (Child et al., 2019; Katharopoulos et al., 2020), Recurrent Neural Networks (RNNs) and State Space Models (SSMs) (Gu et al., 2021; Gu & Dao, 2023; Peng et al., 2023a; De et al., 2024; Feng et al., 2024), sparse attention (Beltagy et al., 2020; Zhao et al., 2019; Xiao et al., 2023; Yuan et al., 2025; Lu et al., 2025) focus on architecture improvements. Chunk strategy have also been explored in long-context modeling (Li et al., 2025b; Liao et al., 2025), while MEMAGENT aims to equip memory ability to any backbone model via post-training with standard RL frameworks without heavily changing on architecture.

Memory Mechanism. The Long Short-Term Memory (LSTM) mechanism (Hochreiter & Schmidhuber, 1997) achieved significant success in early NLP tasks, while Neural Turing Machines (Graves et al., 2014) and Memory Networks (Weston et al., 2014) demonstrated how to equip neural networks with memory. Existing memory mechanisms integrated to Transformer models are typically realized by adding external memory modules (Martins et al., 2021; Wu et al., 2020; Behrouz et al., 2024; Bulatov et al., 2023) or external database (Zhong et al., 2024; Lu et al., 2023; Modarressi et al., 2023). Recently, retrieval-augmented memory agent (Fang et al., 2025; Chhikara et al., 2025; Zhou et al., 2025) workflows have attracted the community’s attention. The diffrence between MEMAGENT and other agent is that we use reinforcement learning to enable LLM itself the ability to memorize.

Reinforcement Learning for LLMs. In recent RL studies, the reward signals have gradually shifted from human preferences (Ouyang et al., 2022) or reward models distilled from them (Bai et al., 2022) to rule-based feedback, which has demonstrated great potential in enhancing model reasoning capabilities (OpenAI, 2024; Guo et al., 2025; Qwen, 2024; DeepMind, 2024; Team et al., 2025) with GAE (Schulman et al., 2018) based PPO (Schulman et al., 2017) or GRPO (Shao et al., 2024) training. Algorithmic enhancements (Hu, 2025; Yu et al., 2025; Liu et al., 2025) have mostly focused on improving sustainability and efficiency of these algorithms. To further release the potential of RL, recent works such as Search-R1 (Jin et al., 2025), Agent-R1 (Ouyang et al., 2025) and RAGEN (Wang et al., 2025) have explored the training of tool-using agents based on multi-turn chat. GiGPO (Feng et al., 2025) further investigates the use of multiple independent contexts in agent training.

5 CONCLUSION

In this paper, we introduce MEMAGENT, a novel long-context method that employs an RL-trained memory module, which enables large language models (LLMs) to selectively record relevant information while disregarding extraneous details. Our experiments demonstrate that when trained on 60K-length sequences, MEMAGENT exhibits remarkable extrapolation, extending its effective context to 3.5M tokens with only 8K context. The model achieves state-of-the-art performance across a diverse range of long-context tasks. Our ablation studies reveal the critical role of RL-based training in achieving these results and how memory capacity influences performance across different task types, providing key insights into the proposed memory mechanism. We hope that this work may lay a strong foundation for developing more advanced memory architectures and training strategies, thereby paving the way for significantly enhancing the long-context capabilities of LLMs.

表4: 探测实验结果。 **Ctx. Dist.** 表示上下文分布，其中两个数字对应于两个关键信息组在整个上下文中的相对位置。0%表示开头，100%表示结尾。 **random** 表示随机打乱所有上下文项，与主实验的设置一致。其他行显示了相对于 **random** 的性能差异。所有值表示准确率（%）。

模型	Ctx. Dist.	长度							
		7K	14K	28K	56K	112K	224K	448K	AVG
14B	random	80.47	82.03	82.03	83.59	81.25	77.34	79.69	75.78
	0% 20%	+3.91	-3.91	+3.13	+1.57	0.00	+4.69	+3.90	+1.90
	0% 100%	+3.12	+0.78	+3.13	-3.12	+1.56	+7.82	+6.25	+2.79
	20% 80%	+0.78	-3.12	+2.35	+0.79	-3.13	+5.47	-3.13	0.00
	40% 60%	+1.56	+2.35	-2.34	-3.12	-1.56	+3.13	-0.78	-0.11
	80% 100%	-2.35	0.00	+1.56	+0.79	+3.13	0.00	+1.56	+0.67
7B	随机	81.25	81.25	82.03	80.47	79.69	75.78	76.56	79.58
	0% 20%	-1.56	-0.78	+3.13	+3.91	+3.12	+5.47	+3.13	+2.35
	0% 100%	+0.78	0.00	+2.35	+1.56	+3.90	+4.69	+2.35	+2.23
	20% 80%	-0.78	-0.78	+2.35	0.00	0.00	0.00	+3.13	+0.56
	40% 60%	0.00	+1.56	+3.13	+0.78	-3.91	+3.13	+5.47	+1.45
	80% 100%	+1.56	0.00	+0.78	-0.78	+0.78	+4.69	0.00	+1.00

修改位置嵌入的组件，使模型能够捕获长距离语义依赖。另一方面，线性注意力机制（Child等人, 2019; Katharopoulos等人, 2020），循环神经网络（RNNs）和状态空间模型（SSMs）（Gu等人, 2021; Gu & Dao, 2023; 彭等人, 2023a; De等人, 2024; Feng等人, 2024），稀疏注意力（Beltagy等人, 2020; Zhao等人, 2019; Xiao等人, 2023; Yuan等人, 2025; Lu等人, 2025)专注于架构改进。分块策略也在长上下文建模中得到了探索（李等人, 2025b; Liao等人, 2025），而MEMAGENT旨在通过标准RL框架的微调，为任何主干模型赋予内存能力，而无需大幅改变架构。

记忆机制。长短期记忆（LSTM）机制（Hochreiter & Schmidhuber, 1997）在早期NLP任务中取得了显著成功，而神经图灵机（Graves et al., 2014）和记忆网络（Weston et al., 2014）则展示了如何为神经网络配备内存。现有集成到Transformer模型中的记忆机制通常通过添加外部内存模块（Martins et al., 2021; Wu et al., 2020; Behrouz et al., 2024; Bulatov et al., 2023）或外部数据库（Zhong et al., 2024; Lu et al., 2023; Modarressi et al., 2023）实现。最近，检索增强记忆代理（Fang et al., 2025; Chhikara et al., 2025; Zhou et al., 2025）工作流程引起了社区的关注。MEMAGENT与其他代理的区别在于，我们使用强化学习使LLM本身具备记忆能力。

强化学习用于大语言模型。在最近的强化学习研究中，奖励信号已逐渐从人类偏好（Ouyang 等人, 2022）或从中提炼的奖励模型（Bai 等人, 2022）转向基于规则的反馈，后者在增强模型推理能力方面展现出巨大潜力（OpenAI, 2024; Guo 等人, 2025; Qwen, 2024; DeepMind, 2024; Team 等人, 2025），并结合基于 GAE（Schulman 等人, 2018）的 PPO（Schulman 等人, 2017）或 GRPO（Shao 等人, 2024）训练。算法增强（Hu, 2025; Yu 等人, 2025; Liu 等人, 2025）主要集中于提高这些算法的可持续性和效率。为了进一步释放强化学习的潜力，近期研究如 Search-R1（Jin 等人, 2025）、Agent-R1（Ouyang 等人, 2025）和 RAGEN（Wang 等人, 2025）探索了基于多轮对话的工具使用代理的训练。GiGPO（Feng 等人, 2025）进一步研究了在代理训练中使用多个独立上下文。

5 结论

在本文中，我们介绍了MEMAGENT，这是一种新型长上下文方法，它采用了一个经过强化学习（RL）训练的内存模块，使大语言模型（LLMs）能够选择性地记录相关信息，同时忽略无关细节。我们的实验表明，当在60K长度的序列上训练时，MEMAGENT表现出卓越的外推能力，仅用8K上下文即可将其有效上下文扩展至3.5M个token。该模型在多种长上下文任务中取得了最先进的性能。我们的消融研究表明，基于RL的训练在取得这些结果中起着关键作用，以及记忆容量如何影响不同任务类型下的性能，为所提出的记忆机制提供了关键见解。我们希望这项工作能为开发更高级的记忆架构和训练策略奠定坚实基础，从而为显著增强LLMs的长上下文能力铺平道路。

REPRODUCIBILITY STATEMENT

For reproducibility, we have provided the implementation details in (§ A), including the prompt template (§ A.1), pseudocode (§ A.2) and training recipe and algorithm hyperparameter (§ A.3) and evaluation settings (§ A.4). The training and evaluation code, as well as the dataset and model weights, will be available in open-source platforms.

可复现性声明

为了可复现性，我们提供了实现细节 (§ A)，包括提示模板 (§ A.1)、伪代码 (§ A.2) 以及训练配方和算法超参数 (§ A.3) 和评估设置 (§ A.4)。训练和评估代码，以及数据集和模型权重，将开源平台提供。

REFERENCES

Chenxin An, Fei Huang, Jun Zhang, Shansan Gong, Xipeng Qiu, Chang Zhou, and Lingpeng Kong. Training-free long-context scaling of large language models. *arXiv preprint arXiv:2402.17463*, 2024.

Anthropic. Claude 3.5 sonnet, 2024. URL <https://www.anthropic.com/news/claude-3-5-sonnet>.

Anthropic. Introducing claude 4, 2025. URL <https://www.anthropic.com/news/claude-4>.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3119–3137, 2024.

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

bloc97. NTK-Aware Scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation., 2023. URL https://www.reddit.com/r/LocalLLaMA/comments/141z7j5/ntkaware_scaled_rope_allows_llama_models_to_have/.

Aydar Bulatov, Yuri Kuratov, Yermek Kapushev, and Mikhail S Burtsev. Scaling transformer to 1m tokens and beyond with rmt. *arXiv preprint arXiv:2304.11062*, 2023.

Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. *arXiv preprint arXiv:2105.03011*, 2021.

Soham De, Samuel L Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.

Google DeepMind. Gemini 2.0 flash thinking, 2024. URL <https://deepmind.google/technologies/gemini/flash-thinking/>.

Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025.

Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.

参考文献

陈新, 黄飞, 张俊, 龚珊珊, 邱习鹏, 周昌, 孔令鹏. 无需训练的大语言模型长上下文扩展. *arXiv* 预印本 *arXiv:2402.17463*, 2024年.

Anthropic. Claude 3.5 sonnet, 2024年. URL<https://www.anthropic.com/news/claude-3-5-sonnet>. Anthropic. 推出 claude 4, 2025年. URL<https://www.anthropic.com/news/claude-4>. 白云涛, 卡达瓦斯·萨鲁夫, 昆杜·桑迪潘, 阿曼达·阿斯科尔, 杰克逊·科尼恩, 安德鲁·琼斯, 安娜·陈, 安娜·戈尔迪, 阿扎莉亚·米尔霍西尼, 卡梅隆·麦金农, 等. 宪法人工智能: 从 AI 反馈中获得无害性. *arXiv* 预印本 *arXiv:2212.08073*, 2022年.

白宇石, 吕欣, 张佳杰, 吕洪昌, 唐建凯, 黄志典, 杜正晓, 刘晓, 曾澳航, 侯雷, 等人. Longbench: 一个双语、多任务的长上下文理解基准测试. 在 第62届计算语言学协会年会议论文集 (第一卷: 长论文) 中, 第3119-3137页, 2024.

Ali Behrouz, Peilin Zhong, 和 Vahab Mirrokni. Titans: 在测试时学习记忆. *arXiv* 预印本 *arXiv:2501.00663*, 2024. Iz Beltagy, Matthew E Peters, 和 Arman Cohan. Longformer: 长文档 Transformer. *arXiv* 预印本 *arXiv:2004.05150*, 2020. bloc97. NTK-Aware Scaled RoPE 允许 LLaMA 模型具有扩展的 (8k+) 上下文大小, 无需任何微调和极小的困惑度退化, 2023. URL https://www.reddit.com/r/LocalLLaMA/comments/141z7j5/ntkaware_scaled_rope_allows_llama_models_to_have/. Aydar Bulatov, Yuri Kuratov, Yermek Kapushev, 和 Mikhail S Burtsev. 使用 rmt 将 Transformer 扩展到 1m tokens 及以上. *arXiv* 预印本 *arXiv:2304.11062*, 2023. Shouyuan Chen, Sherman Wong, Liangjian Chen, 和 Yuandong Tian. 通过位置插值扩展大型语言模型的上下文窗口. *arXiv* 预印本 *arXiv:2306.15595*, 2023. Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, 和 Deshraj Yadav. Mem0: 构建具有可扩展长期记忆的生产就绪 AI 代理. *arXiv* 预印本 *arXiv:2504.19413*, 2025. Rewon Child, Scott Gray, Alec Radford, 和 Ilya Sutskever. 使用稀疏 Transformer 生成长序列. *arXiv* 预印本 *arXiv:1904.10509*, 2019. Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, 和 Matt Gardner. 一个信息寻求型问题和答案的数据集, 锚定在研究论文中. *arXiv* 预印本 *arXiv:2105.03011*, 2021. Soham De, Samuel L Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, 等. Griffin: 混合门控线性递归和局部注意力以实现高效语言模型. *arXiv* 预印本 *arXiv:2402.19427*, 2024. Google DeepMind. Gemini 2.0 快速思考, 2024. URL <https://deepmind.google/technologies/gemini/flash-thinking/>. Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, 和 Ningyu Zhang. Memp: 探索代理程序内存. *arXiv* 预印本 *arXiv:2508.06433*, 2025. Lang Feng, Zhenghai Xue, Tingcong Liu, 和 Bo An. 用于 LLM 代理训练的组内组策略优化. *arXiv* 预印本 *arXiv:2505.10978*, 2025.

Leo Feng, Frederick Tung, Hossein Hajimirsadeghi, Mohamed Osama Ahmed, Yoshua Bengio, and Greg Mori. Attention as an rnn. *arXiv preprint arXiv:2405.13956*, 2024.

Chaochen Gao, Xing Wu, Zijia Lin, Debing Zhang, and Songlin Hu. Nextlong: Toward effective long-context training without long documents. *arXiv preprint arXiv:2501.12766*, 2025.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2021.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

Jian Hu. Reinforce++: A simple and efficient approach for aligning large language models. *arXiv preprint arXiv:2501.03262*, 2025.

Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. Efficient attentions for long document summarization. *arXiv preprint arXiv:2104.02112*, 2021.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

Gregory Kamradt. Needle In A Haystack - pressure testing LLMs. *Github*, 2023. URL https://github.com/gkamradt/LLMTest_NeedleInAHaystack/tree/main.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pp. 5156–5165. PMLR, 2020.

Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328, 2018.

Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. Minimax-01: Scaling foundation models with lightning attention. *arXiv preprint arXiv:2501.08313*, 2025a.

Yucheng Li, Bo Dong, Chenghua Lin, and Frank Guerin. Compressing context to enhance inference efficiency of large language models. *arXiv preprint arXiv:2310.06201*, 2023.

Leo Feng, Frederick Tung, Hossein Hajimirsadeghi, Mohamed Osama Ahmed, Yoshua Bengio, 以及 Greg Mori. Attention as an rnn. arXiv preprint arXiv:2405.13956, 2024. Chaochen Gao, Xing Wu, Zijia Lin, Debing Zhang, 以及 Songlin Hu. Nextlong: Toward effective long-context training without long documents. arXiv preprint arXiv:2501.12766, 2025. Alex Graves, Greg Wayne, 以及 Ivo Danihelka. Neural turing machines. arXiv preprint arXiv:1410.5401, 2014. Albert Gu 和 Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752, 2023. Albert Gu, Karan Goel, 以及 Christopher Ré. Efficiently modeling long sequences with structured state spaces. arXiv preprint arXiv:2111.00396, 2021. Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, 等. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025. Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, 以及 Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. arXiv preprint arXiv:2011.01060, 2020. Sepp Hochreiter 和 Jürgen Schmidhuber. Long short-term memory. Neural computation, 9(8): 1735–1780, 1997. Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, 以及 Boris Ginsburg. Ruler: What’s the real context size of your long-context language models? arXiv preprint arXiv:2404.06654, 2024. Jian Hu. Reinforce++: A simple and efficient approach for aligning large language models. arXiv preprint arXiv:2501.03262, 2025. Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, 以及 Lu Wang. Efficient attentions for long document summarization. arXiv preprint arXiv:2104.02112, 2021. Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, 以及 Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. arXiv preprint arXiv:2310.05736, 2023. Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, 以及 Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. arXiv preprint arXiv:2503.09516, 2025. Gregory Kamradt. Needle In A Haystack - pressure testing LLMs. Github, 2023. URL https://github.com/gkamradt/LLMTest_NeedleInAHaystack/tree/main. Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, 以及 François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In International conference on machine learning, pp. 5156–5165. PMLR, 2020. Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, 以及 Edward Grefenstette. The narrativeqa reading comprehension challenge. Transactions of the Association for Computational Linguistics, 6:317–328, 2018. Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, 等. Minimax-01: Scaling foundation models with lightning attention. arXiv preprint arXiv:2501.08313, 2025a. Yucheng Li, Bo Dong, Chenghua Lin, 以及 Frank Guerin. Compressing context to enhance inference efficiency of large language models. arXiv preprint arXiv:2310.06201, 2023.

Zhenyu Li, Yike Zhang, Tengyu Pan, Yutao Sun, Zhichao Duan, Junjie Fang, Rong Han, Zixuan Wang, and Jianyong Wang. Focusllm: Precise understanding of long context by dynamic condensing. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 31087–31101, 2025b.

Zihan Liao, Jun Wang, Hang Yu, Lingxiao Wei, Jianguo Li, and Wei Zhang. E2llm: Encoder elongated large language models for long-context understanding and reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 19212–19241, 2025.

Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

Xiaoran Liu, Hang Yan, Shuo Zhang, Chenxin An, Xipeng Qiu, and Dahua Lin. Scaling laws of rope-based extrapolation. *arXiv preprint arXiv:2310.05209*, 2023.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, et al. Moba: Mixture of block attention for long-context llms. *arXiv preprint arXiv:2502.13189*, 2025.

Junru Lu, Siyu An, Mingbao Lin, Gabriele Pergola, Yulan He, Di Yin, Xing Sun, and Yunsheng Wu. Memochat: Tuning llms to use memos for consistent long-range open-domain conversation. *arXiv preprint arXiv:2308.08239*, 2023.

Pedro Henrique Martins, Zita Marinho, and Andr e FT Martins. ∞ -former: Infinite memory transformer. *arXiv preprint arXiv:2109.00301*, 2021.

George A Miller et al. The magical number seven, plus or minus two. *Psychological review*, 63(2): 81–97, 1956.

Ali Modarressi, Ayyoob Imani, Mohsen Fayyaz, and Hinrich Sch utze. Ret-llm: Towards a general read-write memory for large language models. *arXiv preprint arXiv:2305.14322*, 2023.

OpenAI. GPT4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

OpenAI. Learning to reason with llms, 2024. URL <https://openai.com/index/learning-to-reason-with-llms/>.

Jie Ouyang, Ruiran Yan, Yucong Luo, Mingyue Cheng, Qi Liu, Zirui Liu, Shuo Yu, and Daoyu Wang. Training powerful llm agents with end-to-end reinforcement learning, 2025. URL <https://github.com/0russwest0/Agent-R1>.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 27730–27744. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf.

Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, et al. Rwkv: Reinventing rnns for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023a.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023b.

李振宇, 张奕科, 潘腾宇, 孙宇涛, 段志超, 方俊杰, 韩荣, 王梓璇, 王建永. Focusllm: 通过动态压缩实现长上下文精确理解. 在第63届计算语言学协会年会论文集（第一卷：长论文），pp. 31087–31101, 2025b.

廖志涵, 王俊, 余杭, 魏凌霄, 李建国, 张伟. E2llm: 用于长上下文理解和推理的编码器扩展大型语言模型. 在 第2025届自然语言处理经验方法会议论文集, pp. 19212–19241, 2025.

刘爱欣, 冯北, 王斌, 王冰轩, 刘博, 赵成刚, 邓成奇, 阮崇, 戴大迈, 郭大雅, 等. Deepseek-v2: 一个强大、经济且高效的专家混合语言模型. *arXiv预印本 arXiv:2405.04434*, 2024.

刘晓然, 颜杭, 张硕, 安晨欣, 邱曦鹏, 和林大华. 基于绳索的外推扩展规律. *arXiv 预印本 arXiv:2310.05209*, 2023.

刘子辰, 陈长宇, 李文俊, 齐鹏辉, 庞天宇, 杜超, 李伟森, 和林敏. 理解 r1-zero 类训练：一个关键视角. *arXiv 预印本 arXiv:2503.20783*, 2025.

陆恩哲, 姜哲俊, 刘景元, 杜宇伦, 姜涛, 洪超, 刘少伟, 何伟然, 袁恩明, 王宇之, 等. Moba: 为长上下文大语言模型混合块注意力. *arXiv 预印本 arXiv:2502.13189*, 2025.

陆俊儒, 安思宇, 林明宝, Gabriele Pergola, 何玉兰, 殷迪, 孙行, 和吴云胜. Memochat: 调优大语言模型使用备忘录实现一致的长距离开放域对话. *arXiv 预印本 arXiv:2308.08239*, 2023.

Pedro Henrique Martins, Zita Marinho和Andr e FT Martins. ∞ -前：无限内存转换器. *arXiv预印本arXiv:2109.00301*,2021。

乔治·A·米勒等. 七加减二：神奇的数字. 心理学评论, 63(2): 81–97, 1956.

Ali Modarressi、Ayyoob Imani、Mohsen Fayyaz和Hinrich Sch utze. Ret-llm：面向大语言模型通用读写内存. *arXiv预印本arXiv:2305.14322*, 2023。

OpenAI. GPT4技术报告. *arXiv预印本 arXiv:2303.08774*, 2023.

OpenAI. 学习使用大语言模型进行推理, 2024. URL<https://openai.com/index/学习使用大语言模型进行推理/>.

Jie Ouyang, Ruiran Yan, Yucong Luo, Mingyue Cheng, Qi Liu, Zirui Liu, Shuo Yu, and Daoyu Wang. 使用端到端强化学习训练强大的llm代理, 2025. URL<https://github.com/0russwest0/Agent-R1>.

欧阳龙, Jeffrey Wu, 江旭, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, 张崇, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, 以及Ryan Lowe. 通过人类反馈训练语言模型遵循指令。在S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, 以及A. Oh（编），《神经信息处理系统进展》，第35卷，第27730–27744页。Curran Associates, Inc., 2022。URLhttps://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf。

包鹏, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, 曹欢奇, 程欣, 钟明, Matteo Grella, 等. Rwkv: 为Transformer时代重塑RNN. *arXiv预印本 arXiv:2305.13048*, 2023a。

彭 Bowen, Jeffrey Quesnelle, 范 Honglu, 以及 Enrico Shippole. YaRN：大语言模型的上下文窗口高效扩展. *arXiv 预印本 arXiv:2309.00071*, 2023b。

Qwen. Qwq-32b: Embracing the power of reinforcement learning, 2024. URL <https://qwenlm.github.io/blog/qwq-32b/>.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation, 2018. URL <https://arxiv.org/abs/1506.02438>.

ByteDance Seed, Jiaze Chen, Tiantian Fan, Xin Liu, Lingjun Liu, Zhiqi Lin, Mingxuan Wang, Chengyi Wang, Xiangpeng Wei, Wen yuan Xu, et al. Seed1. 5-thinking: Advancing superb reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.13914*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, YK Li, Y Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.

Fanqi Wan, Weizhou Shen, Shengyi Liao, Yingcheng Shi, Chenliang Li, Ziyi Yang, Ji Zhang, Fei Huang, Jingren Zhou, and Ming Yan. Qwenlong-1l: Towards long-context large reasoning models with reinforcement learning. *arXiv preprint arXiv:2505.17667*, 2025.

Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning, 2025. URL <https://arxiv.org/abs/2504.20073>.

Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. *arXiv preprint arXiv:1410.3916*, 2014.

Qingyang Wu, Zhenzhong Lan, Kun Qian, Jing Gu, Alborz Geramifard, and Zhou Yu. Memformer: A memory-augmented transformer for sequence modeling. *arXiv preprint arXiv:2010.06891*, 2020.

XAI. Grok 3 beta — the age of reasoning agents, 2024. URL <https://x.ai/news/grok-3>.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, et al. Effective long-context scaling of foundation models. *arXiv preprint arXiv:2309.16039*, 2023.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Qwen. Qwq-32b: 拥抱强化学习的力量, 2024。URL<https://qwenlm.github.io/blog/qwq-32b/>。John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, 以及 Oleg Klimov. 近端策略优化算法。*arXiv* 预印本 *arXiv:1707.06347*, 2017。John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, 以及 Pieter Abbeel. 使用广义优势估计的高维连续控制, 2018。URL <https://arxiv.org/abs/1506.02438>。字节跳动种子计划, 陈 Jiaze, 范 Tiantian, 刘 Xin, 刘 Lingjun, 林 Zhiqi, 王 Mingxuan, 王 Chengyi, 魏 Xiangpeng, 徐 Wen yuan, 等。Seed1. 5思考: 通过强化学习提升卓越推理模型的推理能力。*arXiv* 预印本 *arXiv:2504.13914*, 2025。邵 Zhihong, 王 Peiyi, 朱 Qihao, 徐 Runxin, 宋 Junxiao, 张 Mingchuan, 李 YK, 王 Y, 郭 Daya。Deepseekmath: 推动开放语言模型中的数学推理极限。*arXiv* 预印本 *arXiv:2402.03300*, 2024。盛 Guangming, 张 Chi, 叶 Zilingfeng, 吴 Xibin, 张 Wang, 张 Ru, 彭 Yanghua, 林 Haibin, 吴 Chuan。Hybridflow: 一个灵活高效的 RLHF 框架。*arXiv* 预印本 *arXiv:2409.19256*, 2024。苏 Jianlin, Ahmed Murtadha, 陆 Yu, 潘 Shengfeng, 伯 Wen, 刘 Yunfeng。Roformer: 具有旋转位置嵌入的增强型 Transformer。*Neurocomputing*, 568:127063, 2024。Kimi 团队, 杜 Angang, 高 Bofei, 行 Bowei, 江 Changjiu, 陈 Cheng, 李 Cheng, 肖 Chenjun, 杜 Chenzhuang, 廖 Chonghua, 等。Kimi k1. 5: 使用大语言模型的强化学习扩展。*arXiv* 预印本 *arXiv:2501.12599*, 2025。Trivedi Harsh, Balasubramanian Niranjan, Khot Tushar, Sabharwal Ashish。MuSiQue: 通过单跳问题组合实现多跳问题。计算语言学协会会刊, 10:539–554, 2022。万 Fanqi, 沈 Weizhou, 廖 Shengyi, 石 Yingcheng, 李 Chenliang, 杨 Ziyi, 张 Ji, 黄 Fei, 周 Jingren, 严 Ming。QwenLong-1l: 通过强化学习实现长上下文大推理模型。*arXiv* 预印本 *arXiv:2505.17667*, 2025。王 Zihan, 王 Kangrui, 王 Qineng, 张 Pingyue, 李 Linjie, 杨 Zhengyuan, 金 Xing, 于 Kefan, Nguyen Minh Nhat, 刘 Licheng, Gottlieb Eli, 卢 Yiping, Cho Kyunghyun, 吴 Jiajun, Fei-Fei Li, 王 Lijuan, Choi Yejin, 李 Manling。RAGEN: 通过多轮强化学习理解大语言模型代理的自我进化, 2025。URL <https://arxiv.org/abs/2504.20073>。Weston Jason, Chopra Sumit, Bordes Antoine。记忆网络。*arXiv* 预印本 *arXiv:1410.3916*, 2014。吴 Qingyang, 兰 Zhenzhong, 钱 Kun, 顾 Jing, Geramifard Alborz, 周 Zhou。Memformer: 用于序列建模的记忆增强 Transformer。*arXiv* 预印本 *arXiv:2010.06891*, 2020。XAI。Grok 3 beta — 推理代理的时代, 2024。URL<https://x.ai/news/grok-3>。肖 Guangxuan, 田 Yuandong, 陈 Beidi, 韩 Song, Lewis Mike。具有注意力陷阱的高效流式语言模型。*arXiv* 预印本 *arXiv:2309.17453*, 2023。熊 Wenhan, 刘 Jingyu, Molybog Igor, 张 Hejia, Bhargava Prajjwal, 侯 Rui, Martin Louis, Rungta Rashi, Sankararaman Karthik Abinav, Oguz Barlas, 等。基础模型的有效长上下文扩展。*arXiv* 预印本 *arXiv:2309.16039*, 2023。杨 An, 杨 Baosong, 张 Beichen, 会 Binyuan, 郑 Bo, 于 Bowen, 李 Chengyuan, 刘 Dayiheng, 黄 Fei, 魏 Haoran, 等。Qwen2. 5 技术报告。*arXiv* 预印本 *arXiv:2412.15115*, 2024。

An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiandong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, Junyang Lin, Kai Dang, Kexin Yang, Le Yu, Mei Li, Minmin Sun, Qin Zhu, Rui Men, Tao He, Weijia Xu, Wenbiao Yin, Wenyuan Yu, Xiafei Qiu, Xingzhang Ren, Xinlong Yang, Yong Li, Zhiying Xu, and Zipeng Zhang. Qwen2.5-1m technical report. *arXiv preprint arXiv:2501.15383*, 2025.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. Helmet: How to evaluate long-context language models effectively and thoroughly. *arXiv preprint arXiv:2410.02694*, 2024.

Qiyong Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, YX Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.

Jiaxin Zhang, Yiqi Wang, Xihong Yang, Siwei Wang, Yu Feng, Yu Shi, Ruichao Ren, En Zhu, and Xinwang Liu. Test-time training on graphs with large language models (llms). In *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 2089–2098, 2024.

Guangxiang Zhao, Junyang Lin, Zhiyuan Zhang, Xuancheng Ren, Qi Su, and Xu Sun. Explicit sparse transformer: Concentrated attention through explicit selection. *arXiv preprint arXiv:1912.11637*, 2019.

Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, et al. Qmsum: A new benchmark for query-based multi-domain meeting summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5905–5921, 2021.

Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*, 2025.

杨安, 余 Bowen, 李成元, 刘代恒, 黄飞, 黄浩岩, 蒋建东, 涂建红, 张建伟, 周景仁, 林俊阳, 党凯, 杨克欣, 余乐, 李梅, 孙敏敏, 朱秦, 门瑞, 何涛, 徐伟嘉, 尹文标, 余文远, 邱夏菲, 任兴章, 杨新龙, 李勇, 徐志英, 张子鹏. Qwen2.5-1m技术报告。*arXiv*预印本 *arXiv:2501.15383*, 2025年。

杨志林, 戚鹏, 张赛争, 约书亚·本吉奥, 威廉·W·科恩, 拉尔斯兰·萨拉赫乌丁诺夫, 以及克里斯托弗·D·曼宁。热点问答: 一个用于多样化、可解释的多跳问答数据集。*arXiv*预印本 *arXiv:1809.09600*, 2018年。

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, 以及Danqi Chen。头盔: 如何有效且全面地评估长上下文语言模型。*arXiv*预印本 *arXiv:2410.02694*, 2024年。

Qiyong Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, 等。DAPO: 一个大规模开源LLM强化学习系统。*arXiv*预印本 *arXiv:2503.14476*, 2025年。

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, YX Wei, Lean Wang, Zhiping Xiao, 等。原生稀疏注意力: 与硬件对齐的原生可训练稀疏注意力。*arXiv*预印本*arXiv:2502.11089*, 2025年。

张嘉欣, 王一奇, 杨希红, 王思伟, 冯宇, 石宇, 任瑞超, 朱恩, 刘新旺。基于大语言模型 (llms) 的图上测试时训练。在 第 32 届 ACM 国际多媒体会议论文集, 第 2089–2098 页, 2024。

赵广祥, 林俊阳, 张志远, 任宣成, QiSu, 和 孙旭。显式稀疏 Transformer: 通过显式选择集中注意力。*arXiv* 预印本 *arXiv:1912.11637*, 2019。

钟明, 尹达, 余涛, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan, Asli Celikyilmaz, 刘杨, 邱习鹏, 等。Qmsum: 一个新的基于查询的多领域会议摘要基准。在 第 2021 届北美洲计算语言学协会分会会议: 人机语言技术, 第 5905–5921 页, 2021。

钟万军, 郭良红, 高琪琪, 叶鹤, 和王延林。Memorybank: 增强大型语言模型的长期记忆。在《AAAI人工智能会议论文集》第38卷, 第19724–19731页, 2024年。

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, 以及Paul Pu Liang。Mem1: 学习协同内存和推理以实现高效的长期智能体。*arXiv*预印本*arXiv:2506.15841*, 2025。

A IMPLEMENTATION DETAILS

A.1 PROMPT TEMPLATE

You are presented with a problem, a section of an article that may contain the answer, and a previous memory. Please read the section carefully and <i>update the memory</i> with new information that helps to answer the problem, while retaining all relevant details from the previous memory. <pre><problem> {prompt} </problem> <memory> {memory} </memory> <section> {chunk} </section> Updated memory:</pre> You are presented with a problem and a previous memory. Please answer the problem based on the previous memory and put the answer in \boxed {}. <pre><problem> {prompt} </problem> <memory> {memory} </memory> Your answer:</pre>

Table 5: Template of MEMAGENT for context processing (top part) and final answer generation (bottom). Curly-brace placeholders {} will be replaced with actual content.

A.2 ALGORITHM

Algorithm 1 Multi-conv DAPO

Require: Policy model π_θ , reference model π_{ref} (frozen), rule-based verifier V , dataset $\mathcal{D}$, group size G

- 1: **while** not converged **do**
- 2: Sample a prompt $q \sim \mathcal{D}$
- 3: **for** $g = 1$ to G **do** ▷ Group of conversations for the same q
- 4: Initialize $h_{g,0} \leftarrow [q]$
- 5: **for** $t = 1$ to c_g **do** ▷ Context-independent conversation
- 6: Sample $o_{g,t} \sim \pi_\theta(\cdot \mid h_{g,t-1})$
- 7: $h_{g,t} \leftarrow h_{g,t-1} \parallel o_{g,t}$
- 8: **end for**
- 9: $y_g \leftarrow o_{g,c_g}$ ▷ Final response used for scoring
- 10: $\hat{r}_g \leftarrow V(q, y_g)$ ▷ Rule-based reward
- 11: $d_g \leftarrow \text{KL}(\pi_\theta(\cdot \mid h_{g,c_g}) \parallel \pi_{\text{ref}}(\cdot \mid h_{g,c_g}))$
- 12: $r_g \leftarrow \hat{r}_g - \beta d_g$
- 13: **end for**
- 14: $\{A_g\}_{g=1}^G \leftarrow \text{GroupNorm}(\{r_g\}_{g=1}^G)$
- 15: **for** $g = 1$ to G **do**
- 16: $\rho_g \leftarrow \frac{\pi_\theta(y_g \mid h_{g,c_g})}{\pi_{\theta_{\text{old}}}(y_g \mid h_{g,c_g})}$
- 17: $\mathcal{J}_g \leftarrow \min(\rho_g A_g, \text{clip}(\rho_g, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) A_g)$
- 18: **end for**
- 19: $\mathcal{J}_{\text{clip}} \leftarrow \frac{1}{G} \sum_{g=1}^G \mathcal{J}_g$
- 20: $\theta \leftarrow \theta + \eta \nabla_\theta \mathcal{J}_{\text{clip}}$
- 21: **end while**

A.3 TRAINING

We use the DAPO algorithm for training, applying a KL factor of 1×10^{-3} and disabling the entropy loss. The AdamW optimizer is employed with a constant learning rate of 1×10^{-6} and a linear warm-up scheduler, with the warm-up step set to 20. We use a rollout batchsize of 256, with a group size of 16. Note that due to the multi-conversation feature of MEMAGENT, the actual mini-batchsize

A. 实现细节

A.1 提示模板

您面前呈现了一个 **问题**，一篇文章的 **段落**，该段落可能包含答案，以及一个 **之前的内存**。请仔细阅读该段落，并用有助于回答问题的新的信息来 **更新内存**，同时保留之前内存中的所有相关细节。

<问题> {提示} </问题><内存> {内存} </内存><段落> {片段} </段落>**更新后的内存：**您面前呈现了一个 **问题** 和一个 **之前的内存**。请根据之前的内存回答问题，并将答案放在 \boxed {} 中。<问题> {提示} </问题><内存> {内存} </内存>**您的答案：**

表5: MEMAGENT用于上下文处理（上部）和最终答案生成（下部）的模板。花括号占位符 {} 将被实际内容替换。

A.2 算法

算法1多卷积DAPO

需要： 策略模型 π_θ 、参考模型 π_{ref} (Frozen)、基于规则的验证器 V 、数据集 $\mathcal{D}$ 、组大小 G

- 1: **当** 未收敛**时**
- 2: 采样一个提示 $q \sim \mathcal{D}$
- 3: **for** $g = 1$ 到 G **do** ▷ 相同 q 的对话组
- 4: 初始化 $h_{g,0} \leftarrow [q]$
- 5: **for** $t = 1$ 到 c_g **do** ▷ 上下文无关对话
- 6: 样本 $o_{g,t} \sim \pi_\theta(\cdot \mid h_{g,t-1})$
- 7: $h_{g,t} \leftarrow h_{g,t-1} \parallel o_{g,t}$
- 8: **结束for循环**
- 9: $y_g \leftarrow o_{g,c_g}$ ▷ 用于评分的最终响应
- 10: $\hat{r}_g \leftarrow V(q, y_g)$ ▷ 基于规则的奖励
- 11: $d_g \leftarrow \text{KL}(\pi_\theta(\cdot \mid h_{g,c_g}) \parallel \pi_{\text{ref}}(\cdot \mid h_{g,c_g}))$
- 12: $r_g \leftarrow \hat{r}_g - \beta d_g$
- 13: **结束for循环**
- 14: $\{A_g\}_{g=1}^G \leftarrow \text{组归一化}(\{r_g\}_{g=1}^G)$
- 15: **for** $g = 1$ 到 G **do**
- 16: $\rho_g \leftarrow \frac{\pi_\theta(y_g \mid h_{g,c_g})}{\pi_{\theta_{\text{old}}}(y_g \mid h_{g,c_g})}$
- 17: $\mathcal{J}_g \leftarrow \min(\rho_g A_g, \text{CLIP}(\rho_g, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) A_g)$
- 18: **结束for循环**
- 19: $\mathcal{J}_{\text{clip}} \leftarrow \frac{1}{G} \sum_{g=1}^G \mathcal{J}_g$
- 20: $\theta \leftarrow \theta + \eta \nabla_\theta \mathcal{J}_{\text{clip}}$
- 21: **结束循环**

A.3 训练

我们使用DAPO算法进行训练，应用KL因子为 1×10^{-3} 并禁用熵损失。采用AdamW优化器，设置恒定学习率为 1×10^{-6} ，并使用线性预热调度器，预热步长设为20。我们使用256的滚动批大小，组大小为16。请注意，由于MEMAGENT的多对话特性，实际的小批量大小并不等于滚动批大小/16。我们通过固定样本批大小与反向传播批大小的比例进行离线策略训练，该比例设置为16。

is not equal to `rollout_batchsize/16`. We utilize off-policy training by fixing the ratio of the sample batch size to the backpropagation batch size is set to 16.

We shift to stage II when stage I are fully converged, which takes about 400 steps. Here is the training data recipe of each stage.

- **Stage I** We use 32,768 synthetic QA data instances, each approximately 32K tokens in length. These are based on the HotpotQA (Yang et al., 2018) dataset and follow the RULER (Hsieh et al., 2024) methodology, which involves embedding golden paragraphs (containing correct answers) within extensive distractor content sampled from the same dataset.
- **Stage II** We use 2,560 training instances with a maximum length of 60K tokens. This set consists of difficult, high-quality long-text QA data from DocQA-RL-1.6K (Wan et al., 2025), mixed with data from the first stage.

Each training sample used in stage I is of 200 articles in HotpotQA, with an approximate total token length of 28K. We thoroughly clean the dataset by filtering out questions where Qwen2.5-7B-Base or Qwen2.5-7B-Instruct achieves 100 % Best-Of-2 score **without given any context**. These questions likely represent common knowledge already internalized within the models’ memories. 80,000 samples from the HotpotQA training split are processed through this pipeline and approximately 50% of the data are filtered out. We chose the frist 32,768 samples of processed data as our training set.

We then apply a similar approach to synthesize 128 samples from the HotpotQA validation set. For extrapolation performance testing, we synthesize test sets with different context lengths using the same pipeline. The number of wiki items ranges from 50 up to 6400, corresponding to context lengths of approximately 7K to 3.5M tokens.

A.4 EVALUATION

We extract answers from the model outputs using regular expressions, and we prompt the model to respond in the specified format. The chosen format is `'the answer is ANSWER.'`

We employ the `sub_em` score for all benchmarks. This means that an answer is considered correct if it contains all the elements of the ground truth. When an answer consists of multiple parts and the expected response should include all of them, the score corresponds to the proportion of correct parts provided.

Before evaluating the answers, we normalize both the ground truth and the extracted responses. For example, we remove definite articles, ignore case distinctions, and apply similar standard normalization steps following previous work (Wan et al., 2025; Hsieh et al., 2024; Yen et al., 2024).

Table 6 shows the generation configurations of baseline models.

Table 6: Generation configurations of baseline models.

Model	Context Length	Input/Output Tokens
QwenLong-L1 (Wan et al., 2025)	128K	120,000 / 10,000
Qwen2.5-Instruct-1M Series (Yang et al., 2025)	1M	990,000 / 10,000
DeepSeek-R1-Distill-Qwen Series (Guo et al., 2025)	128K	120,000 / 10,000
Qwen2.5-Instruct Series(Yang et al., 2024)	128K	120,000 / 10,000

NIAH `niah_single_{1,2,3}` in RULER (Hsieh et al., 2024) benchmark are used in our test. The yaml configuration of RULER are presented in 7. In level 1, the ”haystack” consists of repetitive sentences, and the ”needle” is a seven-digit number associated with a magic word. For level 2, the ”haystack” is composed of longer essays. Level 3 goes a step further than Level 2 where the ”needle” is a 36-character UUID string. Question and context are concated as the input of LLMs. We omit the `answer_prefix` provided in original RULER benchmark since it is not compatible with MEMAGENT workflow.

我们通过固定样本批大小与反向传播批大小的比例进行离线策略训练，该比例设置为16。

当第一阶段完全收敛时，将切换到第二阶段，这大约需要400步。以下是训练数据的配方。

- **第一阶段** 我们使用32,768个合成QA数据实例，每个实例的长度约为32K个token。这些基于热点问答（Yang等人,2018）数据集，并遵循RULER（Hsieh等人, 2024）方法，该方法涉及将包含正确答案的金色段落嵌入从同一数据集中采样的广泛干扰内容中。
- **第二阶段** 我们使用2,560个训练实例，最大长度为60K个token。这个集合包含来自DocQA-RL-1.6K（Wan等人,2025）的困难、高质量的长期文本QA数据，并与第一阶段的数据混合。

第一阶段使用的每个训练样本均来自热点问答数据集，包含200篇文章，总token长度约为28 K。我们通过过滤掉Qwen2.5-7B-Base或Qwen2.5-7B-Instruct在未提供任何上下文的情况下就达到了100%的Best-Of-2分数的问题，来彻底清理数据集。这些问题很可能代表了模型记忆中已经内化的常识。从热点问答训练集的80,000个样本通过该流程处理后，大约有50%的数据被过滤掉。我们选择了处理后的前32,768个样本作为我们的训练集。

然后，我们采用类似的方法从热点问答验证集中合成128个样本。为了进行外推性能测试，我们使用相同的流程合成具有不同上下文长度的测试集。维基百科条目的数量范围从50到6400，对应上下文长度约为7K到3.5M个token。

A.4 评估

我们使用正则表达式从模型输出中提取答案，并提示模型为指定格式中。所选格式是“答案是ANSWER。”

我们采用子 `em` 分数对所有基准进行评估。这意味着如果答案包含所有地面实况元素，则被视为正确。当答案由多个部分组成，而预期响应应包含所有这些部分时，分数对应于所提供正确部分的比例。

在评估答案之前，我们规范化ground truth和提取的响应。例如，我们移除限定词，忽略大小写差异，并应用与先前工作（Wan等人,2025; Hsieh等人, 2024; Yen等人, 2024）类似的标准化步骤。

表 6 展示了基线模型的生成配置。

表 6：基线模型的生成配置。

模型	上下文长度	输入/输出 token
QwenLong-L1 (Wan 等人, 2025)	128K	120,000 / 10,000
Qwen2.5-Instruct-1M 系列（杨等人, 2025）	1M	990,000 / 10,000
DeepSeek-R1-Distill-Qwen 系列（郭等人, 2025）	128K	120,000 / 10,000
Qwen2.5-Instruct 系列（杨等人, 2024）	128K	120,000 / 10,000

NIAH `niah_单一_{1,2,3}` 在RULER (Hsieh等人, 2024)基准测试中用于我们的测试。RULER的yaml配置在<样式id='8'>7</样式>中呈现。在第一级，”干草堆”由重复的句子组成，而”针”是一个与魔法词关联的七位数。在第二级，”干草堆”由更长的文章组成。第三级比第二级更进一步，其中”针”是一个36字符的UUID字符串。问题和上下文被连接作为LLM的输入。我们省略了原始RULER基准中提供的答案前缀，因为它与MEMAGENT工作流程不兼容。

```
niah.single.1:
task: niah
args:
  type.haystack: repeat
  type.needle.k: words
  type.needle.v: numbers
  num.needle.k: 1
  num.needle.v: 1
  num.needle.q: 1
niah.single.2:
task: niah
args:
  type.haystack: essay
  type.needle.k: words
  type.needle.v: numbers
  num.needle.k: 1
  num.needle.v: 1
  num.needle.q: 1
niah.single.3:
task: niah
args:
  type.haystack: essay
  type.needle.k: words
  type.needle.v: uuids
  num.needle.k: 1
  num.needle.v: 1
  num.needle.q: 1
```

Table 7: Synthetic Configuration used for NIAH task.

B COMPUTATION COMPLEXITY

We adopt the floating-point operations (FLOP) estimator for the Qwen2Model from verl [Sheng et al. \(2024\)](#) to compute the FLOP cost of both the baseline model and our proposed method. The results are shown in Figure 10. The baseline model exhibits an $O(n^2)$ complexity, while MEMAGENT achieves an $O(n)$ complexity.

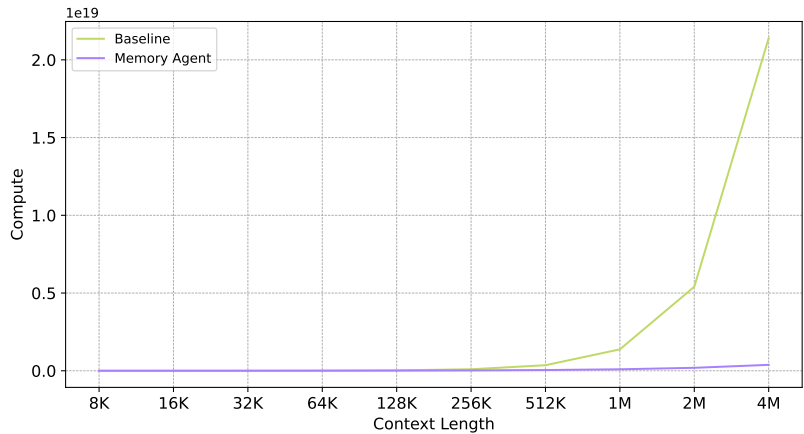


Figure 10: Floating point operations across context lengths from 8K to 4M

For the baseline model, the number of tokens required to process is $q + c + o$, where q represents the length for the problem, c is the context length and o represents the output length.

```
niah单一1: -
任务: niah
参数:
  类型haystack: 重复
  类型needle.k: 单词
  类型needle.v: 数字
  数字针.k: 1
  数字针.v: 1
  数字针.q: 1
niah单个2: -
任务: niah
参数:
  类型haystack: 文章
  类型needle.k: 单词
  类型needle.v: 数字
  数字针.k: 1
  数字针.v: 1
  数字针.q: 1
NIAH 单个 3:-
任务: niah
参数:
  类型haystack: 文章
  类型needle.k: 单词
  类型needle.v: UUID
  数字针.k: 1
  数字针.v: 1
  数字针.q: 1
```

表 7: 用于 NIAH 任务的合成配置。

B 计算复杂度

我们采用verl[Sheng等人\(2024\)](#)的浮点运算(FLOP)估计器来计算基线模型和我们的提出方法的FLOP成本。结果如图<样式id='7'>10</样式>所示。基线模型表现出 $O(n^2)$ 复杂度，而MEMAGENT实现了 $O(n)$ 复杂度。

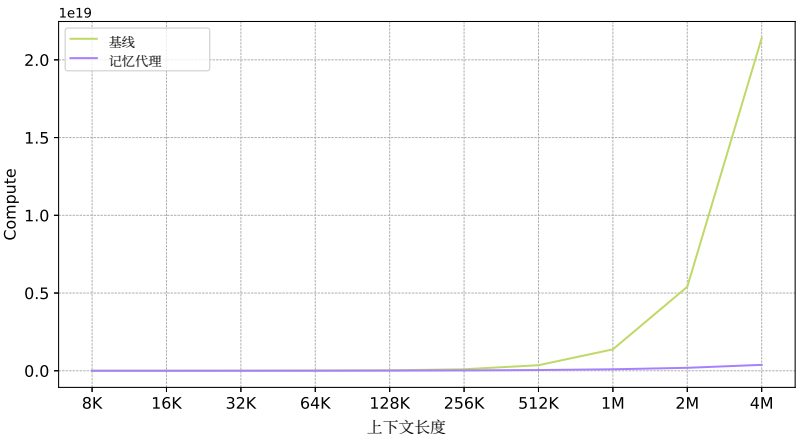


图10: 浮点运算在不同上下文长度（从8K到4M）下的表现

对于基线模型，处理所需的 token 数量为 $q + c + o$ ，其中 q 表示问题长度， c 是上下文长度， o 表示输出长度。

For MEMAGENT, total FLOP cost is the sum of the FLOPs from all stages. The detailed stages involved are as follows:

- **Initializing:** In the first stage, the model processes an input consisting of $q + 200 + o$, where 200 represents a constant added to prompt the model to follow the MEMAGENT workflow.
- **Memory Updating:** The number of repetitions is determined by $k = \lceil \frac{c}{N} \rceil$, where c is the variable component of the input. Each repetition requires an input of length $q + 200 + N + o$.
- **Final Answering:** The final stage processes an input of length $q + 100 + o$, which includes the accumulated output from the previous steps.

We set $q = 1024$, $o = 1024$, $N = 5000$ and c is ranging from 8K to 4M to calculate the final result.

C CASE STUDY

To further illustrate the proposed memory mechanism in detail, we conduct a case study on a generation trajectory of MEMAGENT-14B. The input question is: *The director of the romantic comedy ‘Big Stone Gap’ is based in what New York city?* This a 2-hop question with the following relevant Wikipedia entries:

1) **Big Stone Gap** is a 2014 American drama romantic comedy film written and directed by Adriana Trigiani.

2) **Adriana Trigiani** is an Italian American best-selling author of sixteen books, television writer, film director, and entrepreneur based in Greenwich Village, New York City.

In the first round, the model is presented with the entry *Ghost*, which refers to a production team also based in *New York*. The model chooses to retain this potentially useful information for future use. In the second round, no relevant context is provided; nevertheless, the model maintains its agent state, demonstrating robustness against distraction. In the third round, both relevant entries are presented. The model correctly identifies critical information and updates its memory accordingly, leading to the correct answer: *Greenwich Village, New York City*. At this point, the reasoning process is complete. In the remaining rounds, the model’s memory remains unchanged and is used to produce the final response.

CHUNK 1

...

Document 20: Ghost (production team)

Ghost is a Swedish record producing and songwriting team, composed of Ulf Lindström and Johan Ekhé, based in *New York City*.

...

MEMORY 1

The provided section does not contain any information about the director of the romantic comedy "Big Stone Gap" or their base in New York City. **However, it does mention a production team called "Ghost" which is based in New York City.** Since the problem is specifically asking about the director of "Big Stone Gap," and there is no direct information about the director in the given section, we cannot infer anything directly related to the problem from the provided text.

Updated memory:

- Ghost is a Swedish record producing and songwriting team, composed of Ulf Lindström and Johan Ekhé, based in New York City. They are perhaps best known for writing and producing Swedish singer Robyn’s three first studio albums.

The problem remains unanswered based on the provided section. Additional information would be needed to determine the director of "Big Stone Gap" and their base in New York City.

对于 MEMAGENT, 总 FLOP 成本是所有阶段 FLOP 的总和。涉及的详细阶段如下:

- **初始化:** 在第一阶段, 模型处理一个包含 $q + 200 + o$ 的输入, 其中200代表一个常数, 用于提示模型遵循MEMAGENT代理工作流程。
- **内存更新:** 重复次数由 $k = \lceil \frac{c}{N} \rceil$ 决定, 其中 c 是输入的可变部分。每次重复需要输入长度为 $q + 200 + N + o$ 的数据。
- **最终回答:** 最后阶段处理长度为 $q + 100 + o$ 的输入, 该输入包含先前步骤累积的输出。

We 设置 $q = 1024$ 、 $o = 1024$ 、 $N = 5000$, 并且 c 的范围从8K到4M以计算最终结果。

C 案例研究

为了更详细地说明所提出的记忆机制, 我们对MEMAGENT-14B的一个生成轨迹进行了案例研究。输入问题是: 浪漫喜剧《*Big Stone Gap*》的导演位于哪个纽约市? 这是一个2跳问题, 以下是与问题相关的维基百科条目:

1) **Big Stone Gap** 是一部2014年美国剧情浪漫喜剧电影, 由Adriana Trigiani编剧和导演。

2) **Adriana Trigiani** 是一位意大利裔美国作家, 已出版十六本书籍, 同时也是电视编剧、电影导演和企业家, 总部位于纽约市格林威治村。

在第一轮中, 模型被提供条目*Ghost*, 该条目指的是一个同样位于纽约的制作团队。模型选择保留这条可能有用的信息以供将来使用。在第二轮中, 没有提供相关上下文; 尽管如此, 模型维持了其代理状态, 展示了其对干扰的鲁棒性。在第三轮中, 两个相关条目都被提供。模型正确识别了关键信息并相应地更新了其内存, 从而得出了正确答案: 纽约市格林威治村。此时, 推理过程已结束。在剩余的轮次中, 模型的内存保持不变, 并用于生成最终响应。

片段1

...

文档20: Ghost (制作团队) Ghost是一个瑞典的唱片制作和歌曲创作团队, 由Ulf Lindström和Johan Ekhé组成, 总部位于纽约市。

...

内存1The

所提供的段落并未包含关于浪漫喜剧《Big Stone Gap》导演的信息, 或其位于纽约市的总部。然而, 它确实提到了一个名为“Ghost”的制作团队, 该团队总部位于纽约市。由于问题特别询问的是《Big Stone Gap》的导演, 并且所提供的段落中没有关于导演的直接信息, 因此我们无法从提供的文本中直接推断出任何与问题相关的内容。

更新后的内存:

- Ghost是一个瑞典的唱片制作和歌曲创作团队, 由Ulf Lindström和Johan Ekhé组成, 总部位于纽约市。他们或许最出名的是为瑞典歌手Robyn创作并制作了她的前三张录音室专辑。

根据提供的段落, 该问题仍未得到解答。需要更多信息才能确定《Big Stone Gap》的导演及其在纽约市的基地。

CHUNK2
...(No relevant document)

MEMORY2
The provided section does not contain any information about the director of the romantic comedy "Big Stone Gap" or their base in New York City. 2 Therefore, the updated memory remains unchanged with respect to the problem at hand. The memory still notes that the Ghost record producing and songwriting team is based in New York City, but it does not provide any information about the director of "Big Stone Gap."
Updated memory:

- Ghost is a Swedish record producing and songwriting team, composed of Ulf Lindström and Johan Ekhé, based in New York City. They are perhaps best known for writing and producing Swedish singer Robyn's three first studio albums.
- The problem remains unanswered based on the provided section. Additional information would be needed to determine the director of "Big Stone Gap" and their base in New York City.

CHUNK3
...
Document 81: Big Stone Gap (film)
Big Stone Gap is a 2014 American drama romantic comedy film written and directed by Adriana Trigiani and produced by Donna Gigliotti for Altar Identity Studios, a subsidiary of Media Society.
...
Document 96: Adriana Trigiani
Adriana Trigiani is an Italian American best-selling author of sixteen books, television writer, film director, and entrepreneur based in Greenwich Village, New York City.
...
MEMORY3
Updated memory:

- Ghost is a Swedish record producing and songwriting team, composed of Ulf Lindström and Johan Ekhé, based in New York City. They are perhaps best known for writing and producing Swedish singer Robyn's three first studio albums.
- Adriana Trigiani is an Italian American best-selling author of sixteen books, television writer, film director, and entrepreneur based in Greenwich Village, New York City. Trigiani has published a novel a year since 2000.
- Big Stone Gap is a 2014 American drama romantic comedy film written and directed by Adriana Trigiani and produced by Donna Gigliotti for Altar Identity Studios, a subsidiary of Media Society. Based on Trigiani's 2000 best-selling novel of the same name, the story is set in the actual Virginia town of Big Stone Gap circa 1970s. The film had its world premiere at the Virginia Film Festival on November 6, 2014.
The director of the romantic comedy "Big Stone Gap" is Adriana Trigiani, who is based in Greenwich Village, New York City.

This trajectory highlights several core capabilities learned by the memory agent: estimating potentially relevant content from query keywords and storing it preemptively, immediately updating memory upon encountering context that matches the query, and remaining unaffected by irrelevant information. Notably, these memory behaviors are not the result of architectural attention mechanisms, but emerge as text generation abilities reinforced through RL.

CHUNK2
...（无相关文件）

M内存2Th
ed段落不包含任何关于浪漫喜剧《Big Stone Gap》导演或他们在纽约市基地的信息。2因此，更新的内存当前问题方面保持不变。内存仍然记录了制作和创作团队Ghost的基地在纽约市，但它没有提供关于《Big Stone Gap》导演的任何信息。更新后的内存：

- Ghost是一个瑞典制作和创作团队，由Ulf Lindström和Johan Ekhé组成，基地在纽约市。他们或许最著名的是为瑞典歌手Robyn创作和制作了她的前三张录音室专辑。
- 根据提供的段落，该问题仍未得到解答。需要更多信息才能确定《Big Stone Gap》的导演及其在纽约市的基地。

CHUNK3
...
文件81：《Big Stone Gap》（电影）Big Stone Gap 是一部2014年美国剧情浪漫喜剧电影，由AdrianaTrigiani 编剧和导演，由Donna Gigliotti为Altar Identity Studios制作，后者是Media Society的子公司。
...
文件96：Adriana Trigiani
Adriana Trigiani 是一位意大利裔美国畅销书作家，著有十六本书籍，电视编剧、电影导演和企业家，总部位于纽约市格林威治村。
...
MEMORY3
Updated 内存：

- Ghost是一个瑞典的唱片制作和创作团队，由Ulf Lindström和Johan Ekhé组成，总部位于纽约市。他们或许最出名的是为瑞典歌手Robyn创作并制作了她的前三张录音室专辑。
- Adriana Trigiani 是一位意大利裔美国畅销书作家，著有十六本书籍，电视编剧、电影导演和企业家，总部位于纽约市格林威治村。自2000年起，Trigiani每年出版一部小说。
- Big Stone Gap 是一部2014年美国剧情浪漫喜剧电影，由Adriana Trigiani编剧和导演，Donna Gigliotti为Altar Identity Studios（Media Society的子公司）制作。根据Trigiani2000年畅销同名小说改编，故事背景设定在1970年代实际位于弗吉尼亚州的Big Stone Gap小镇。该电影于2014年11月6日在弗吉尼亚电影节全球首映。
浪漫喜剧《Big Stone Gap》的导演是Adriana Trigiani，她位于纽约市格林威治村。

这条轨迹突出了记忆代理学习到的几项核心能力：根据查询关键词估计潜在的相关内容并预先存储，在遇到与查询匹配的上下文时立即更新内存，以及不受无关信息的影响。值得注意的是，这些记忆行为并非源于架构上的注意力机制，而是作为通过强化学习强化的文本生成能力而涌现的。

D SUPPLEMENTARY EXPERIMENTS

D.1 ABLATION STUDY ON CONTEXT SIZE AND MEMORY SIZE

In this ablation experiments, we keep the total length of memory size + context chunk size constant, and linearly adjusted the memory size. The goal is to control the total context length per conversation turn. The results are shown in Table 8 and Table 9.

Table 8: Ablation result of memory-size and context-size on NIAH. All number indicates the averaging score of level1level3

Method	8K	16K	32K	64K	128K	256K	512K
14B-m4096-c1928	100.00	99.22	98.70	97.66	98.96	99.22	97.92
14B-m3072-c2952	99.74	100.00	100.00	100.00	98.70	99.48	99.48
14B-m2048-c3976	99.74	99.48	99.48	100.00	100.00	97.13	97.66
14B-m1024-c5000	99.74	98.70	100.00	100.00	98.96	97.40	98.18
7B-m4096-c1928	100.00	99.22	98.44	96.35	98.96	98.18	97.14
7B-m3072-c2952	99.48	100.00	99.74	99.48	96.88	98.18	96.09
7B-m2048-c3976	99.74	99.48	99.48	99.48	98.96	94.53	94.53
7B-m1024-c5000	100.00	98.70	99.74	98.96	97.40	97.66	96.62

Table 9: Ablation result of memory-size and context-size on Longbench-QA

Method	2Wiki	HQA	MuSiQue	NQA	Qasper	AVG
14B-m4096-c1928	74.5	72.5	48.5	21.5	25.5	48.5
14B-m3072-c2952	76.5	70.5	52.5	24.5	26.5	50.1
14B-m2048-c3976	74.5	71.5	49.5	23.0	27.0	49.1
14B-m1024-c5000	79.0	73.0	52.0	25.0	26.0	51.0
7B-m4096-c1928	70.0	66.0	45.5	19.0	26.0	45.3
7B-m3072-c2952	72.0	64.0	44.0	20.0	25.5	45.1
7B-m2048-c3976	75.0	69.0	43.5	23.0	26.0	47.3
7B-m1024-c5000	74.0	69.5	47.0	21.5	29.0	48.2

D.2 AGENT BASELINES

We compare MemAgent against an advanced memory-agent method, Mem0(Chhikara et al., 2025). The Mem0 paper also reports that RAG methods using only top-1 or top-2 retrieval form strong and stable baselines for memory-agent tasks. Therefore, we conduct extensive comparisons against RAG agents under multiple configurations.

For Mem0, we use SOTA OpenAI models, GPT-5.1 and text-embedding-3-large as langugae model and embedding model respectively and we follow the official GitHub repository for memory updating and retrieval. Specifically, during memory creation, we split and processed the entire context in 5,000-token chunks; during retrieval, we selected the top 30 memories.

For RAG, we also use text-embedding-3-large as embedding model and configure it with various chunk size and top-K value.

The results show that MEMAGENT outperforms these methods, demonstrating that end-to-end RL-trained memory provides greater flexibility and coherence compared with retrieval-based strategies.

D 补充实验

D.1 关于上下文大小和内存大小的消融研究

在本项消融实验中，我们保持内存大小的总长度和上下文片段大小不变，并线性调整内存大小。目的是控制每轮对话的总上下文长度。结果展示在表 + 中8 和表9.

表 8: NIAH 中内存大小和上下文大小的消融结果。所有数字表示 level1level3 的 ~ 平均得分

方法	8K	16K	32K	64K	128K	256K	512K
14B-m4096-c1928	100.00	99.22	98.70	97.66	98.96	99.22	97.92
14B-m3072-c2952	99.74	100.00	100.00	100.00	98.70	99.48	99.48
14B-m2048-c3976	99.74	99.48	99.48	100.00	100.00	97.13	97.66
14B-m1024-c5000	99.74	98.70	100.00	100.00	98.96	97.40	98.18
7B-m4096-c1928	100.00	99.22	98.44	96.35	98.96	98.18	97.14
7B-m3072-c2952	99.48	100.00	99.74	99.48	96.88	98.18	96.09
7B-m2048-c3976	99.74	99.48	99.48	99.48	98.96	94.53	94.53
7B-m1024-c5000	100.00	98.70	99.74	98.96	97.40	97.66	96.62

表9: 内存大小和上下文大小在Longbench-QA上的消融结果

方法	2Wiki	HQA	MuSiQue	NQA	Qasper	AVG
14B-m4096-c1928	74.5	72.5	48.5	21.5	25.5	48.5
14B-m3072-c2952	76.5	70.5	52.5	24.5	26.5	50.1
14B-m2048-c3976	74.5	71.5	49.5	23.0	27.0	49.1
14B-m1024-c5000	79.0	73.0	52.0	25.0	26.0	51.0
7B-m4096-c1928	70.0	66.0	45.5	19.0	26.0	45.3
7B-m3072-c2952	72.0	64.0	44.0	20.0	25.5	45.1
7B-m2048-c3976	75.0	69.0	43.5	23.0	26.0	47.3
7B-m1024-c5000	74.0	69.5	47.0	21.5	29.0	48.2

D.2 AGENT BASELINES

我们比较MemAgent与一种先进的记忆代理方法Mem0(Chhikara等人, 2025)。Mem0论文还报告说，仅使用top-1或top-2检索的RAG方法为记忆代理任务形成了强大且稳定的基线。因此，我们在多种配置下对RAG代理进行了广泛的比较。

对于Mem0，我们使用SOTA OpenAI模型，GPT-5.1和text-embedding-3-large分别作为语言模型和嵌入模型，并遵循官方GitHub存储库进行内存更新和检索。具体来说，在内存创建过程中，我们将整个上下文分割并处理为5,000-token的片段；在检索过程中，我们选择了前30条记忆。

对于RAG，我们也使用text-embedding-3-large作为嵌入模型，并配置不同的片段大小和top-K值。

结果表明，MEMAGENT 比这些方法表现更优，证明端到端 RL 训练的内存提供了比检索式策略更高的灵活性和连贯性。

Table 10: Result versus RAG Agent in RULER-HQA with different top-**K** settings. We segment the context based on natural semantic units, i.e., each wiki item was treated as a chunk.

Model	Length									
	7K	14K	28K	56K	112K	224K	448K	896K	1.75M	3.5M
RAG + Qwen2.5-14B										
$K=2$	57.03	54.69	51.56	54.69	53.12	50.00	52.34	49.22	48.44	48.44
$K=4$	66.41	67.19	68.75	67.19	66.41	64.06	66.41	64.84	60.94	59.38
$K=6$	72.66	75.78	75.78	74.22	69.53	71.88	73.44	67.19	65.62	66.41
$K=8$	78.12	78.91	77.34	81.25	76.56	78.12	77.34	74.22	70.31	64.84
RL-MEMAGENT-14B	80.47	82.03	82.03	83.59	81.25	77.34	79.69	75.78	78.91	71.09
RAG + Qwen2.5-7B										
$K=2$	53.91	54.69	53.12	51.56	54.69	51.56	52.34	49.22	48.44	46.09
$K=4$	67.19	66.41	66.41	67.19	64.84	64.06	62.50	61.72	60.94	59.38
$K=6$	74.22	73.44	72.66	73.44	70.31	73.44	70.31	67.19	65.62	65.62
$K=8$	75.00	75.00	75.78	74.22	74.22	77.34	72.66	68.75	64.06	64.84
RL-MEMAGENT-7B	81.25	81.25	82.03	80.47	79.69	75.78	76.56	74.22	77.34	71.88

Table 11: Result versus RAG Agent in Longbench-QA with different top-**K** and Context size settings. We segment the context using fixed-length chunks. For retrieval, we performed top-k matching using cosine similarity scores.

Method	2Wiki	HQA	MuSiQue	NQA	Qasper	AVG
Qwen2.5-14B + RAG						
$C=1024 K=2$	51.50	56.50	26.50	15.00	23.50	28.83
$C=1024 K=4$	70.00	64.50	34.50	17.50	27.00	35.58
$C=1024 K=6$	71.50	64.00	41.00	19.00	27.50	37.17
$C=1024 K=8$	72.50	64.50	39.00	17.50	26.00	36.58
$C=2048 K=2$	58.50	61.50	33.50	13.50	25.50	32.08
$C=2048 K=4$	76.00	64.00	36.00	18.50	25.00	36.58
$C=2048 K=6$	73.00	67.50	41.50	21.00	26.50	38.25
$C=2048 K=8$	77.50	68.50	42.00	21.00	27.50	39.42
RL-MemAgent-14B	79.0	73.0	52.00	25.00	26.00	51.00
Qwen2.5-7B + RAG						
$C=1024 K=2$	41.00	48.50	22.00	14.50	25.50	25.25
$C=1024 K=4$	49.00	56.50	28.00	17.00	28.50	29.83
$C=1024 K=6$	54.50	57.50	29.50	17.00	25.00	30.58
$C=1024 K=8$	50.50	59.00	29.50	18.00	25.00	30.33
$C=2048 K=2$	49.50	51.50	19.50	12.50	27.00	26.67
$C=2048 K=4$	50.50	53.00	26.00	17.00	25.50	28.67
$C=2048 K=6$	50.50	56.50	27.50	22.00	25.50	30.33
$C=2048 K=8$	50.50	58.00	25.50	21.00	27.00	30.33
RL-MemAgent-7B	74.00	69.50	47.00	21.50	29.00	48.20

E LLM USAGE

In this section, we report the usage of LLMs in this work. Some sentences in this manuscript are drafted or refined by LLMs, but all text is finalized by human authors. In the experimental process, LLMs assist with code completion, but they do not produce novel ideas or complete experiments.

表10：在RULER-HQA中与RAG代理在不同top-**K** 设置下的结果。我们基于自然语义单元对上下文进行分段，即每个维基百科条目被视为一个片段。

模型	长度									
	7K	14K	28K	56K	112K	224K	448K	896K	1.75M	3.5M
RAG + Qwen2.5-14B										
$K=2$	57.03	54.69	51.56	54.69	53.12	50.00	52.34	49.22	48.44	48.44
$K=4$	66.41	67.19	68.75	67.19	66.41	64.06	66.41	64.84	60.94	59.38
$K=6$	72.66	75.78	75.78	74.22	69.53	71.88	73.44	67.19	65.62	66.41
$K=8$	78.12	78.91	77.34	81.25	76.56	78.12	77.34	74.22	70.31	64.84
RL-MEMAGENT-14B	80.47	82.03	82.03	83.59	81.25	77.34	79.69	75.78	78.91	71.09
RAG + Qwen2.5-7B										
$K=2$	53.91	54.69	53.12	51.56	54.69	51.56	52.34	49.22	48.44	46.09
$K=4$	67.19	66.41	66.41	67.19	64.84	64.06	62.50	61.72	60.94	59.38
$K=6$	74.22	73.44	72.66	73.44	70.31	73.44	70.31	67.19	65.62	65.62
$K=8$	75.00	75.00	75.78	74.22	74.22	77.34	72.66	68.75	64.06	64.84
RL-MEMAGENT-7B	81.25	81.25	82.03	80.47	79.69	75.78	76.56	74.22	77.34	71.88

表11：在Longbench-QA中与RAG代理在不同top-**{K}** 和**{C}**ontextsize设置下的结果对比。我们使用固定长度的片段对上下文进行分段。在检索时，我们使用余弦相似度分数进行top-k匹配。

方法	2Wiki	HQA	MuSiQue	NQA	Qasper	AVG
Qwen2.5-14B + RAG						
$C=1024 K=2$	51.50	56.50	26.50	15.00	23.50	28.83
$C=1024 K=4$	70.00	64.50	34.50	17.50	27.00	35.58
$C=1024 K=6$	71.50	64.00	41.00	19.00	27.50	37.17
$C=1024 K=8$	72.50	64.50	39.00	17.50	26.00	36.58
$C=2048 K=2$	58.50	61.50	33.50	13.50	25.50	32.08
$C=2048 K=4$	76.00	64.00	36.00	18.50	25.00	36.58
$C=2048 K=6$	73.00	67.50	41.50	21.00	26.50	38.25
$C=2048 K=8$	77.50	68.50	42.00	21.00	27.50	39.42
RL-Memagent-14B	79.0	73.0	52.00	25.00	26.00	51.00
Qwen2.5-7B + RAG						
$C=1024 K=2$	41.00	48.50	22.00	14.50	25.50	25.25
$C=1024 K=4$	49.00	56.50	28.00	17.00	28.50	29.83
$C=1024 K=6$	54.50	57.50	29.50	17.00	25.00	30.58
$C=1024 K=8$	50.50	59.00	29.50	18.00	25.00	30.33
$C=2048 K=2$	49.50	51.50	19.50	12.50	27.00	26.67
$C=2048 K=4$	50.50	53.00	26.00	17.00	25.50	28.67
$C=2048 K=6$	50.50	56.50	27.50	22.00	25.50	30.33
$C=2048 K=8$	50.50	58.00	25.50	21.00	27.00	30.33
RL-MemAgent-7B	74.00	69.50	47.00	21.50	29.00	48.20

E LLM 使用

在本节中，我们报告了这项工作中大语言模型的使用情况。本文档中的一些句子由大语言模型起草或润色，但所有文本最终由人类作者确定。在实验过程中，大语言模型协助进行代码补全，但它们不会产生新想法或完成实验。

F FALIURE PATTERN STUDY

F.1 INFORMATION OVERWRITTEN

In this example, the model accumulates a large amount of irrelevant memories in the early stage (Turn 60). When the crucial context appears, the model does capture the relevant information (Turn 289), but attempts to append it to the end (Turn 290), where it is truncated due to insufficient memory. Later, the model proactively performs a summarization (Turn 317), which gives it more space to store the second key piece of context (Turn 433). However, since the information from the first context has already been lost, the model incorrectly relies on irrelevant information for reasoning and fails to answer the question correctly.

Question: Where is the company that Sachin Warriier worked for as a software engineer headquartered?

Relevant Information:
(shown in Turn 289) Tata Consultancy Services Limited (TCS): An Indian multinational information technology (IT) service, consulting and business solutions company. It is a subsidiary of the Tata Group, operates in 46 countries, and is **headquartered in Mumbai, Maharashtra**.
(shown in Turn 433) Sachin Warriier: A playback singer and composer in the Malayalam cinema industry from Kerala. He became notable with the song "Muthuchippi Poloru" from the film Thattathin Marayathu, made his debut with Malarvaadi Arts Club, and was once working as a software engineer at Tata Consultancy Services in Kochi (later resigned to focus on music). His latest work is as a composer for the movie Aanandam.

Answer: Mumbai

Memory Diff between Turn 59 and Turn 60

-The provided section does not contain any information about Sachin Warriier or his work as a software engineer. Therefore, the updated memory remains unchanged with respect to Sachin Warriier. However, it does provide information about Texas Monthly, which is headquartered in Downtown Austin, Texas. Since the problem is specifically asking about a company where Sachin Warriier worked as a software engineer, and no information about Sachin Warriier is provided, the updated memory remains as follows:
+The provided section does not contain any information about Sachin Warriier or his work as a software engineer. However, it does provide information about Procter & Gamble, which is an American consumer goods corporation headquartered in downtown Cincinnati, Ohio, United States of America. Since the problem is specifically asking about a company where Sachin Warriier worked as a software engineer, and no information about Sachin Warriier is provided, the updated memory remains as follows:

- Pizza Pizza Ltd. is a franchised Canadian pizza fast-food restaurant, with its headquarters in Etobicoke, Toronto. Its restaurants are mainly in the province of Ontario. Other locations operate in Quebec, Nova Scotia, and in western Canada. Franchises in western Canada are mostly run through Alberta-based subsidiary Pizza 73, and in non-traditional locations such as university campuses and movie theatres throughout Canada. It has over 500 locations, including over 150 non-traditional locations.
- TI Automotive develops, manufactures, and supplies automotive fluid storage, carrying, and delivery systems. The company has 23,000 employees at 120 locations in 29 countries, and supplies all of the world’s major automobile manufacturers. TI Automotive serves the automotive aftermarket through Bundy, Walbro, and Marwal brands. The company’s headquarters are located in Oxford, UK, with Corporate Offices based in Auburn Hills, Michigan. In 2015 TI Automotive was acquired by Bain Capital.

(...)

• -

F 失败模式研究

F.1 信息被覆盖

在这个例子中，模型在早期阶段（回合60）积累了大量无关的内存。当关键上下文出现时，模型确实捕获了相关信息（回合289），但试图将其附加在末尾（回合290），但由于内存不足而被截断。后来，模型主动执行了摘要（回合317），这给它提供了更多空间来存储第二个关键上下文（回合433）。然而，由于第一个上下文的信息已经丢失，模型错误地依赖无关信息进行推理，并未能正确回答问题。

问题：萨钦·瓦里尔作为软件工程师曾就职的公司总部在哪里？

相关信息：
(在回合 289 中显示) 塔塔咨询服务有限公司（TCS）：一家印度跨国信息技术（IT）服务、咨询和商业解决方案公司。它是塔塔集团（Group）的子公司，在 46 个国家运营，其**总部位于孟买**

Maharashtra.
(显示于回合433) 萨钦·瓦里尔：来自喀拉拉邦的马拉雅拉姆电影产业中的配音歌手和作曲家。他因电影《Thattathin Marayathu》中的歌曲“Muthuchippi Poloru”而闻名，通过Malarvaadi Arts Club出道，曾担任过在科钦塔塔咨询服务的软件工程师（后来辞职专注于音乐）。他的最新工作是电影《Aanandam》的作曲家。

Answer: 孟买

记忆差异 between Turn 59 and Turn 60

-所提供的段落不包含任何关于萨钦·瓦里尔或他作为软件工程师的工作信息。因此，关于萨钦·瓦里尔的更新内存保持不变。然而，它确实提供了关于《德克萨斯月刊》的信息，该杂志总部位于德克萨斯州奥斯汀市市中心。由于问题是专门询问萨钦·瓦里尔曾作为软件工程师工作过的公司，并且没有提供关于萨钦·瓦里尔的信息，更新后的内存保持如下：
+所提供的段落不包含任何关于萨钦·瓦里尔或他作为软件工程师的工作信息。然而，它确实提供了关于宝洁公司的信息，宝洁公司是一家总部位于美国俄亥俄州辛辛那提市市中心的美国消费品公司。由于问题是专门询问萨钦·瓦里尔曾作为软件工程师工作过的公司，并且没有提供关于萨钦·瓦里尔的信息，更新后的内存保持如下：

- Pizza Pizza Ltd. 是一家加拿大特许经营的比萨快餐连锁店，总部位于多伦多的埃托比克。其餐厅主要分布在安大略省。其他门店位于魁北克、新斯科舍省以及加拿大西部。加拿大西部的特许经营店主要通过位于阿尔伯塔省的子公司 Pizza 73 运营，此外还有在加拿大各大学校园和电影院等非传统地点的门店。该公司拥有超过500家门店，其中包括超过150家非传统门店。
- TI Automotive开发、制造和供应汽车液体存储、运输和配送系统。该公司在29个国家的120个地点拥有23,000名员工，并向全球所有主要汽车制造商供货。TI Automotive通过Bundy、Walbro和Marwal品牌为汽车售后市场提供服务。该公司的总部位于英国牛津，企业总部位于密歇根州奥本山。2015年，TI Automotive被贝恩资本收购。

(...)

• -

Memory Diff between Turn 282 and Turn 289

-The provided section does not contain any information about Sachin Warrier or the company he worked for as a software engineer. Therefore, the updated memory remains as follows:

+The provided section contains information about Tata Consultancy Services Limited (TCS), which is an Indian multinational information technology (IT) service, consulting and business solutions company. It states that TCS is headquartered in **Mumbai, Maharashtra**.

+Updated memory:

- Cosmos (formerly Cosmos Tours) is a UK independent tour operator providing a range of package holidays to the UK market. The tour operator is connected to the international Globus Travel Group, founded in 1928, which remains family owned with headquarters in Lugano, Switzerland. Under parent company Cosmos Tours Ltd, the tour operator offers holidays through three brands: Cosmos, Archers Holidays and Avalon Waterways.
- L&F Jones Holdings Ltd is a company based in Westfield, Somerset, England, that includes a convenience store chain, a wholesale food business, and a Best Western PLUS hotel. The store chain comprises twelve shops in Somerset, Wiltshire, Dorset, South Gloucestershire, and Bristol.
(...)
- Round Table Pizza is a large chain of pizza parlors in the western United States. The company’s headquarters are located in Concord, California.
- Round Table Pizza is a large chain of pizza parlors in the western United States. The company’s
- Anton Media Group, formerly Anton Community

Memory Diff between Turn 289 and Turn 290

-The provided section contains information about Tata Consultancy Services Limited (TCS), which is an Indian multinational information technology (IT) service, consulting and business solutions company. It states that TCS is headquartered in **Mumbai, Maharashtra**.

+The provided section does not contain any information about Sachin Warrier or Tata Consultancy Services Limited (TCS). Therefore, the updated memory remains as follows:

-Updated memory:

- Cosmos (formerly Cosmos Tours) is a UK independent tour operator providing a range of package holidays to the UK market. The tour operator is connected to the international Globus Travel Group, founded in 1928, which remains family owned with headquarters in Lugano, Switzerland. Under parent company Cosmos Tours Ltd, the tour operator offers holidays through three brands: Cosmos, Archers Holidays and Avalon Waterways.
- L&F Jones Holdings Ltd is a company based in Westfield, Somerset, England, that includes a convenience store chain, a wholesale food business, and a Best Western PLUS hotel. The store chain comprises twelve shops in Somerset, Wiltshire, Dorset, South Gloucestershire, and Bristol.
(...)
- Tata Consultancy Services Limited (TCS), an Indian multinational information technology (

回合 282 与回合 289 的记忆差异

-所提供的段落不包含任何关于萨钦·瓦里尔或他曾作为软件工程师就职的公司信息。因此，更新的内存保持如下：+所提供的段落包含关于塔塔咨询服务有限公司（TCS）的信息，这是一家印度跨国信息技术（IT）服务、咨询和商业解决方案公司。它指出TCS总部位于**孟买，马哈拉施特拉邦**。+更新后的内存：

- Cosmos（前身为Cosmos Tours）是一家英国独立旅行社，为英国市场提供一系列包年假期。该旅行社隶属于成立于1928年的国际Globus旅游集团，该集团至今仍为家族所有，总部位于瑞士卢加诺。在母公司Cosmos Tours有限公司旗下，该旅行社通过三个品牌提供假期：Cosmos、Archers Holidays和Avalon Waterways。
- L&F Jones Holdings Ltd是一家总部位于英国索默塞特西菲尔德的公司，旗下包括一家便利店连锁店、一家批发食品业务和一家最佳西方PLUS酒店。该连锁店在索默塞特、威尔特郡、多塞特、南格洛斯特郡和布里斯托尔拥有十二家门店。
(...)
- Round Table Pizza是美国西部一家大型比萨连锁店。该公司总部位于加利福尼亚州康科德。
- Round Table Pizza 是美国西部一家大型比萨餐厅连锁店。该公司的
- Anton Media Group, 前身为Anton社区

记忆差异 between 回合 289 and 回合 290

-所提供的段落包含有关塔塔咨询服务有限公司（TCS）的信息，它是一家印度跨国信息技术（IT）服务、咨询和商业解决方案公司。它指出TCS总部位于**孟买，马哈拉施特拉邦**。+所提供的段落不包含有关萨钦·瓦里尔或塔塔咨询服务有限公司（TCS）的任何信息。因此，更新的内存保持如下：-更新后的内存：

- Cosmos（前身为Cosmos Tours）是一家英国独立旅行社，为英国市场提供一系列包年假期。该旅行社隶属于成立于1928年的国际Globus旅游集团，该集团至今仍为家族所有，总部位于瑞士卢加诺。在母公司Cosmos Tours有限公司旗下，该旅行社通过三个品牌提供假期：Cosmos、Archers Holidays和Avalon Waterways。
- L&F Jones控股有限公司是一家总部位于西菲尔德，索默塞特，英国的公司，旗下包括一家便利店连锁店、一家批发食品业务和一家最佳西方PLUS酒店。该连锁店在索默塞特、威尔特郡、多塞特、南格洛斯特郡和布里斯托尔拥有十二家店铺。
(...)
- 塔塔咨询服务有限公司（TCS），一家印度跨国信息技术（

Memory Diff between Turn 316 and Turn 317

The provided section does not contain any information relevant to Sachin Warriier or Tata Consultancy Services Limited (TCS). Therefore, the updated memory remains as follows:

- (formerly Cosmos Tours) is a UK independent tour operator providing a range of package holidays to the UK market. The tour operator is connected to the international Globus Travel Group, founded in 1928, which remains family owned with headquarters in Lugano, Switzerland. Under parent company Cosmos Tours Ltd, the tour operator offers holidays through three brands: Cosmos, Archers Holidays and Avalon Waterways.
- L&F Jones Holdings Ltd is a company based in Westfield, Somerset, England, that includes a convenience store chain, a wholesale food business, and a Best Western PLUS hotel. The store chain comprises twelve shops in Somerset, Wiltshire, Dorset, South Gloucestershire, and Bristol.
- (...)
- The company’s corporate headquarters is in Faridabad, Haryana, India.
- Diageo plc (or) is a British multinational alcoholic beverages company, with its headquarters in London, England. It
- (...)

Since the problem is specifically about the headquarters of the company Sachin Warriier worked for as a software engineer, and the previous memory already states that Tata Consultancy Services Limited (TCS) is headquartered in Faridabad, Haryana, India, the updated memory remains as follows:

- The company’s corporate headquarters is in Farid

Memory Diff between Turn 430 and Turn 433

-The provided section does not contain any information about Sachin Warriier or his employment as a software engineer. Therefore, the updated memory remains as follows:

+The provided section contains information about Sachin Warriier, which is useful for answering the problem. Specifically, it states:

+ Sachin Warriier is a playback singer and composer in the Malayalam cinema industry from Kerala.

+ He was working as a software engineer in Tata Consultancy Services in Kochi.

+ Later he resigned from the job to concentrate more on music.

+However, the section does not provide the headquarters information for Tata Consultancy Services. Given the previous memory, we know that the corporate headquarters of Tata Consultancy Services is in Faridabad, Haryana, India. Therefore, the updated memory is as follows:

- The company’s corporate headquarters is in Faridabad, Haryana, India.
- SpaceX is an American aerospace manufacturer and space transport services company headquartered in Hawthorne, California.
- (...)
- Sachin Warriier’s company headquarters information is still unknown from the given documents.
- Sachin Warriier was working as a software engineer in Tata Consultancy Services in Kochi. Later he resigned from the job to concentrate more on music.
- AVG Technologies is a security software company headquartered in Amsterdam, Netherlands.
- (...)

-The updated memory reflects that the headquarters of the companies mentioned are in Faridabad, Haryana, India; Hawthorne, California; Amsterdam, Netherlands; Eagan, Minnesota; Pittsburgh, Pennsylvania; and San Antonio, Texas. However, there is no information about the company Sachin Warriier worked for as a software engineer. Therefore, the memory remains unchanged in terms of Sachin Warriier’s company headquarters information.

+The updated memory confirms that Sachin Warriier worked as a software engineer for Tata Consultancy Services in Kochi, and the headquarters of Tata Consultancy Services is in Faridabad, Haryana, India.

回合316与回合317的记忆差异

所提供的段落不包含任何与萨钦·瓦里尔或塔塔咨询服务有限公司（TCS）相关的信息。因此，更新的内存保持如下：

- （前身为Cosmos Tours）是一家英国独立旅行社，为英国市场提供一系列包年假期。该旅行社隶属于成立于1928年的国际Globus旅游集团，该集团至今仍为家族所有，总部位于瑞士卢加诺。在母公司Cosmos Tours Ltd旗下，该旅行社通过三个品牌提供假期：Cosmos、Archers Holidays和Avalon Waterways。
- L&F Jones控股有限公司是一家总部位于英国索默塞特西菲尔德的公司，旗下包括一家便利店连锁店、一家批发食品业务和一家最佳西方PLUS酒店。该连锁店在索默塞特、威尔特郡、多塞特、南格洛斯特郡和布里斯托尔拥有十二家门店。
- (...)
- 该公司总部位于印度哈里亚纳邦法里达巴德
- 迪吉欧公司（或迪吉欧）是一家总部位于英国伦敦的英国跨国酒精饮料公司。它(…)

由于问题特别涉及萨钦·瓦里尔作为软件工程师曾为之工作的公司总部，并且之前的记忆已经说明塔塔咨询服务有限公司（TCS）总部位于印度哈里亚纳邦法里达巴德，因此更新的记忆保持如下： - 该公司的企业总部位于法里德

记忆差异（第430回合与第433回合之间）

-所提供的段落并未包含关于萨钦·瓦里尔或其作为软件工程师的就业信息。因此，更新的记忆保持如下： +所提供的段落包含关于萨钦·瓦里尔的信息，这对解决问题有用。具体来说，它说明： + 萨钦·瓦里尔是来自喀拉拉邦的马拉雅拉姆电影产业中的回放歌手和作曲家。+ 他曾在塔塔咨询服务公司科钦担任软件工程师。+ 后来他辞去了工作，以便更专注于音乐。

+然而，该段落并未提供塔塔咨询服务公司的总部信息。根据之前的记忆，我们知道塔塔咨询服务公司的企业总部位于印度哈里亚纳邦法里达巴德。因此，更新的记忆如下：

- 该公司的企业总部位于印度哈里亚纳邦法里达巴德。
- SpaceX是一家总部位于加利福尼亚州霍索恩的美国航空航天制造商和太空运输服务公司。(…)
- 从提供的文档中，萨钦·瓦里尔的总部信息仍然未知。
- 萨钦·瓦里尔曾在塔塔咨询服务公司在科钦担任软件工程师。后来他辞去了这份工作，以便更专注于音乐。
- AVG Technologies是一家总部位于荷兰阿姆斯特丹的安全软件公司。(…)

-更新的内存反映，所提及公司的总部位于印度哈里亚纳邦法里达巴德；加利福尼亚州霍索恩；荷兰阿姆斯特丹；明尼苏达州埃agan；宾夕法尼亚州匹兹堡；以及德克萨斯州圣安东尼奥。然而，关于萨钦·瓦里尔作为软件工程师曾就职的公司总部信息没有提供。因此，在萨钦·瓦里尔公司总部信息方面，内存保持不变。

+更新的内存确认，萨钦·瓦里尔曾在塔塔咨询服务公司担任软件工程师，而塔塔咨询服务公司的总部位于印度哈里亚纳邦的法里达巴德。

F.2 MISSING CRITICAL INFORMATION

In this example, when the model encounters the first key piece of information (Turn 143), it fails to recognize it as such because this is a multi-hop question and the prerequisite evidence has not yet been observed. Consequently, even though the model becomes aware of the problem upon seeing the second piece of information (Turn 215), it still fails to produce the correct final answer.

Question: who is the younger brother of The episode guest stars of The Hard Easy ?

Relevant Information:
(shown in Turn 143) Brian Doyle-Murray Brian Doyle-Murray (born Brian Murray, October 31, 1945) is an American actor, voice artist, comedian and screenwriter. **He is the older brother of actor/comedian Bill Murray**, and the two have acted together in several films...
(shown in Turn 215) The Hard Easy (Adventure Time) ”The Hard Easy” is the twenty-third episode...
The episode guest stars Brian Doyle-Murray as Prince Huge and Jonathan Katz as the Mudscamp elder.

Answer: Bill Murray

Memory Diff between Turn 142 and Turn 143

-The provided section does not contain any information related to ”The Hard Easy” or its guest stars. Therefore, the updated memory remains as follows:
+The provided section does not contain any information related to ”The Hard Easy” or its guest stars. However, it does provide information about Brian Doyle-Murray, who is the older brother of actor/comedian Bill Murray. Since the problem is asking for the younger brother of the episode guest stars of ”The Hard Easy,” and the memory does not contain any information about the guest stars of ”The Hard Easy,” the information about Brian Doyle-Murray and Bill Murray is not directly relevant to the problem at hand.
+Therefore, the updated memory remains as follows:

- (...)
- Martin Lee Truex Jr. (born June 29, 1980) is an American championship professional stock car racing driver. His younger brother, Ryan, is a champion in the K&N Pro Series East division and currently races in the Camping World Truck Series.
(...)

Memory Diff between Turn 185 and Turn 215

-The provided section does not contain any information related to ”The Hard Easy” or its guest stars. Therefore, the updated memory remains unchanged with respect to the problem at hand. The only relevant piece of information is that the younger brother of Martin Lee Truex Jr. is Ryan Truex.
+The provided section contains information about ”The Hard Easy,” an episode of ”Adventure Time,” which **includes the guest stars Brian Doyle-Murray as Prince Huge and Jonathan Katz as the Mudscamp elder. However, it does not provide any information about the younger brother of the episode guest stars.**

-Thus, the updated memory is as follows:
+The updated memory is as follows:

- ...
- Martin Lee Truex Jr. (born June 29, 1980) is an American championship professional stock car racing driver. His younger brother, Ryan, is a champion in the K&N Pro Series East division and currently races in the Camping World Truck Series. (...)

The younger brother of Martin Lee Truex Jr. is Ryan Truex.

+The provided section does not contain any information about the younger brother of the guest stars of ”The Hard Easy.” Therefore, the updated memory remains unchanged with respect to the problem at hand. The only relevant piece of information is that the younger brother of Martin Lee Truex Jr. is Ryan Truex.

F.2 丢失关键信息

在这个例子中，当模型遇到第一个关键信息（第143回合）时，它无法识别其为关键信息，因为这是一个多跳问题，先决证据尚未观察到。因此，尽管模型在看到第二个信息（第215回合）时意识到了问题，它仍然无法给出正确的最终答案。

问题：《硬核易》的客串嘉宾中，谁是弟弟？

相关信息：
(出现在第 143 轮) 布莱恩·道尔·默里 布莱恩·道尔·默里（生于 1945 年 10 月 31 日）是一位美国演员、配音演员、喜剧演员和编剧。 **他是演员/喜剧演员比利·默里的哥哥**，两人曾在多部电影中共同出演... (出现在第 215 轮) 《硬核易》（冒险时间） “《硬核易》是第 23 集节目... **该集客串布莱恩·道尔·默里饰演大力王，乔纳森·卡茨饰演泥怪长老。**

答案： 比利·默里

记忆差异 between Turn 142 and Turn 143

-所提供的段落不包含任何与“硬易”或其客串演员相关的信息。因此，更新的内存保持如下：
+所提供的段落不包含任何与“硬易”或其客串演员相关的信息。然而，它确实提供了关于布莱恩·道尔·默里（Brian Doyle-Murray）的信息，他是演员/喜剧演员比尔·默里（Bill Murray）的哥哥。由于问题是要求“硬易”中 episode 客串演员的弟弟，而内存中不包含任何关于“硬易”客串演员的信息，因此关于布莱恩·道尔·默里和比尔·默里的信息与当前问题不直接相关。
+因此，更新的内存保持如下：

- (...)
- 马丁·李·特鲁克斯（小）（1980年6月29日出生）是一位美国冠军职业股票汽车赛车手。他的弟弟瑞安，是K&N专业系列东部分区的冠军，目前在参加露营世界卡车系列赛。
(...)

记忆差异 between Turn 185 and Turn 215

-所提供的段落不包含任何与“硬是容易”或其客串演员相关的信息。因此，更新的内存相对于当前问题保持不变。唯一相关的信息是马丁·李·特拉克斯中尉的弟弟是瑞安·特拉克斯。
+所提供的段落包含有关“硬是容易”的信息，这是“探险时光”的一集，其中 **包括布莱恩·道伊尔·默里客串扮演巨大的王子，乔纳森·卡茨客串扮演泥营长者。但是，它没有提供任何有关这些客串演员弟弟的信息。**

-因此，更新后的内存如下： +更新后的内存如下：

- ...
- 马丁·李·特拉克斯（小）（1980年6月29日出生）是一位美国冠军职业赛车车手。他的弟弟瑞安在K&N职业系列东部赛区是冠军，目前参加坎普林世界卡车系列赛。(…)

马丁·李·特鲁克斯Jr的弟弟是瑞安·特鲁克斯。

+所提供的段落并未包含有关《硬核易》客串演员的弟弟的信息。因此，更新的内存内容与当前问题无关，保持不变。唯一相关的信息是马丁·李·特鲁克斯小弟的弟弟是瑞安·特鲁克斯。

F.3 PRIMACY BIAS

This example involves a primacy bias. The word *country* can refer either to a nation or to the countryside. The model initially assumes it refers to a nation (Turn 1) and subsequently reinforces this belief when it encounters additional information about countries (Turn 617). As a result, even though it eventually also attends to the truly critical piece of information (Turn 728), it still fails to answer the question correctly.

Question: Brown State Fishing Lake is in a country that has a population of how many inhabitants ?

Relevant Information:
(shown in Turn 728) Brown County, Kansas Brown County (county code BR) is a county located in the northeast portion of the U.S. state of Kansas. As of the 2010 census, the county population was 9,984. Its county seat and most populous city is Hiawatha. Brown County is the location of the Kickapoo Indian Reservation of Kansas, the majority of the Sac and Fox Reservation and the majority of the Iowa Reservation of Kansas and Nebraska.

Answer: 9,984

Memory Diff between Turn 0 and Turn 1

-The provided section does not contain any information about the population of the country where Brown State Fishing Lake is located. However, it does confirm that Brown State Fishing Lake is in Brown County, Kansas in the United States.

+The provided section does not contain any information about the population of the United States or any country. Therefore, the updated memory remains as follows:

- Brown State Fishing Lake is a protected area in Brown County, Kansas in the United States.
- The lake is 62 acres (0.25 km²) in area and up to 13 feet (4 m) deep. (...)

-Since the problem is asking for the population of the country where Brown State Fishing Lake is located, and the only country mentioned in the text is the United States, we can infer that the question is seeking the population of the United States. However, the text does not provide the exact population number.

-Given the information available, the updated memory should reflect the following:

+Since the problem is seeking the population of the country where Brown State Fishing Lake is located, and the only country mentioned is the United States, the updated memory reflects that the population of the United States is the required information. However, the text does not provide the exact population number. Therefore, the updated memory retains the above information without changes related to the population.

Memory Diff between Turn 616 and Turn 617

-The provided section contains information about Kunming, a city in Yunnan Province, Southwest China, which has a population of 6,626,000 as of 2014. However, this information is not directly related to the problem at hand, which is about the population of the country where Brown State Fishing Lake is located.

+... Since the problem is asking for the population of the country where Brown State Fishing Lake is located, and we know that Brown State Fishing Lake is in Brown County, Kansas, in the United States, we can infer that the population of the United States is the answer to the problem.

-The updated memory remains consistent with the previous information, but it now includes the new information about Kunming:

+Updated memory:

- Brown State Fishing Lake is a protected area in Brown County, Kansas in the United States.
- The lake is 62 acres (0.25 km²) in area and up to 13 feet (4 m) deep. (...)
- Ruzhou, a city in Henan province, China, has more than 100,000 inhabitants. (...)
- As of 2018, the United States had a population of around 327,167,434.

+The problem is asking for the population of the country where Brown State Fishing Lake is located, which is the United States. Therefore, the population of the United States is around 327,167,434 as of 2018.

F.3 优先效应

这个例子涉及首因效应。单词 *country* 既可以指一个国家，也可以指乡村。模型最初假设它指的是一个国家（第1轮），并在后续遇到更多关于国家的信息时（第617轮）强化了这一信念。结果，尽管它最终也关注了真正关键的信息（第728轮），但仍未能正确回答问题。

问题: Brown 州立钓鱼湖位于一个有多少人口的国家?

相关信息:
(显示在第 728 轮) Brown 县, 堪萨斯州 Brown 县 (县代码 BR) 位于美国堪萨斯州东北部。根据 2010 年人口普查, 该县人口为 9,984 人。其县治和最大城市是 Hiawatha。Brown 县是堪萨斯州 Kickapoo 印第安保留地、大部分 Sac 和 Fox 保留地以及大部分堪萨斯州和内布拉斯加州爱荷华保留地的所在地。

答案: 9,984

记忆差异 (回合0与回合1之间)

-所提供的段落未包含关于布朗州钓鱼湖所在国家的任何信息。然而, 它确实证实了布朗州钓鱼湖位于美国堪萨斯州的布朗县。+所提供的段落未包含关于美国或任何国家的任何信息。因此, 更新的内存保持如下:

- 布朗州钓鱼湖位于美国堪萨斯州的布朗县, 是一个受保护区域。
- 该湖面积达62英亩 (0.25平方公里²), 最深处达13英尺 (4米)。(…)

-由于问题是询问布朗州钓鱼湖所在国家的总人口, 而文本中仅提及了美国, 因此可以推断问题是在寻求美国的总人口。然而, 文本并未提供确切的人口数字。

-根据现有信息, 更新的内存应反映如下: +由于问题是寻求布朗州钓鱼湖所在国家的总人口, 而文本中仅提及了美国, 因此更新的内存反映了美国总人口是所需信息。然而, 文本并未提供确切的人口数字。因此, 更新的内存保持上述信息, 与人口相关的部分不做更改。

记忆差异 (轮次616与轮次617之间)

-所提供的段落包含有关昆明 (中国云南省的一个城市) 的信息, 该城市2014年人口为 662,600。然而, 这些信息与当前问题不直接相关, 当前问题是关于布朗州钓鱼湖所在国家的总人口。

+...由于问题是询问布朗州钓鱼湖所在国家的总人口, 而我们知道布朗州钓鱼湖位于美国堪萨斯州的布朗县, 因此我们可以推断美国的总人口是问题的答案。

-更新的内存与之前的信息保持一致, 但现在包含了关于昆明的最新信息: +更新后的内存:

- 布朗州钓鱼湖是位于美国堪萨斯州布朗县的一个保护区, 美国。
- 这个湖的面积是62英亩 (0.25平方公里²), 最深处达13英尺 (4米)。(…)
- 驻马店, 中国河南省的一个城市, 人口超过10万人。(…)
- 截至2018年, 美国的人口约为327,167,434。

+这个问题要求的是布朗州钓鱼湖所在国家的总人口, 即美国。因此, 截至2018年, 美国的人口约为327,167,434。

Memory Diff between Turn 654 and Turn 728

The provided section does not contain any information relevant to the problem at hand. Therefore, the updated memory remains as follows:

- Brown State Fishing Lake is a protected area in Brown County, Kansas in the United States. (...)
- Brown County, Kansas, as of the 2010 census, had a population of 9,984. (...)
- As of 2018, the United States had a population of around 327,167,434.

The problem is asking for the population of the country where Brown State Fishing Lake is located, which is the United States. Therefore, the population of the United States is around 327,167,434 as of 2018.

Turn 654 与 Turn 728 的记忆差异

所提供的段落不包含与当前问题相关的任何信息。因此，更新的内存保持如下：

- 布朗州钓鱼湖是位于美国堪萨斯州布朗县的一个保护区。(...)
- 根据2010年人口普查，布朗县的人口为9,984。(...)
- 截至2018年，美国的人口约为327,167,434。

问题是询问布朗州钓鱼湖所在国家的 population，即美国。因此，截至2018年，美国的人口约为327,167,434。